



Big Data : Outils et méthodes





Avant de comment :

Présentons nous

- Moi :
 - Formation
 - Expérience professionnelle
 - Vous :
 - Formation
 - Expérience professionnelle actuelles (service de provenance et métier)
 - Niveau de connaissances en databases, NoSQL, Big data, docker, Cloud
 - Attentes par rapport à ce cours
- => *Pour faire simple :*
- Se connecter sur <https://github.com/ababacaryoro/formation-bigdata/> pour voir le README
 - se connecter au tableau https://miro.com/app/board/uXiVH4b8L9M= et mettre Nom, Prénom + informations demandées
 - Ou mettre dans la discussion de la reunion Teams



Objectives

- 1 : Big data : fondamentaux et calcul parallèle
- 2 : Architectures, containerisation et NoSQL
- 3 : Traitement distribué avec Spark
- 4 : Pipelines et orchestration
- 5 : Analyse & déploiement



Objective 1 : Big data - fondamentaux et calcul parallèle

- Big Data et statistique publique
- Mise en place et limites du poste de travail
- Concepts du calcul distribué
- Formats de données massives



Objective 2 : Architectures, containerisation et NoSQL

- Architectures Big Data
- Initiation à Docker
- Découverte de MongoDB
- TP MongoDB : application via un projet



Objective 3 : Traitement distribué avec Spark

- Introduction à Spark
- Projet avec PySpark
- Appariement de sources massives : techniques et application
- Optimisations sur Spark & application



Objective 4 : Pipelines et orchestration

- Traitement en temps réel avec Kafka
- Orchestration avec Airflow
- TP d'intégration : raccordement de bout en bout



Objective 5 : Analyse & Déploiement

- Analyse avancée sur données massives avec Spark
- Visualisation avec les outils de dashboarding



Méthodes d'évaluation





Evaluations

- Plusieurs TP au fil de l'eau
- Projet final à présenter



Mode d'organisation

- => Vous pouvez **travailler en groupe**
- => Les **TP** sont à déposer sur votre **compte github**
- => Vous pouvez **partager** vos connaissances, c'est une session d'échange
- => Essayez de **comprendre** au moins la **structure du code** trouvé sur internet ou via une IA avant de passer



Part 1 :

Big data : Fondamentaux et
calcul parallèle



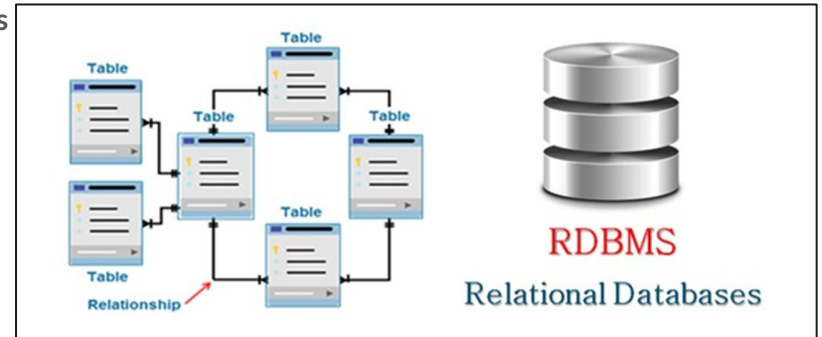
1.1

Big Data et statistique publique

Évolution de la donnée

Données structurées

- **Avant : données structurées**
 - Enquêtes et recensements, collecte maîtrisée
 - La **génération** de la donnée est **principalement provoquée**
 - Organisation en **tables** (lignes et colonnes)
 - Stockées dans des **bases de données relationnelles**
 - Traitement fait avec **SQL** dans des SGBD
 - Exemples de SGBD:
 - MySQL
 - PostgreSQL
 - Oracle





Évolution de la donnée

Données structurées

- **Avant : données structurées**
 - Quel système de stockage à l'ANSD ?
 - Quels types de données ?
 - Quelle taille (GB, dimensions lignes X colonnes) ?



Des données structurées au Big data

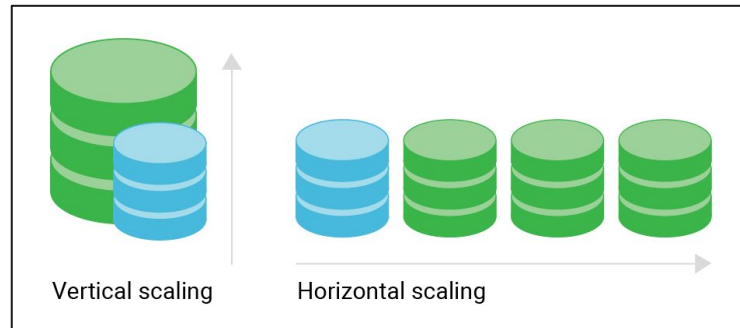
Données structurées

- **Avantages des SGBD relationnels :**
 - **Schéma strict**
 - Type de données : String, Integer, Boolean, Date, ...
 - Relations : PK, Unique, Non NULL, FK, ...
 - Facile à manipuler avec **SQL**
 - Transactions **ACID** :
 - **Atomicité** : Une transaction doit être exécutée entièrement ou pas du tout ; tout ou rien.
 - **Cohérence** : Une transaction doit faire passer la base de données d'un état valide à un autre état valide, en respectant toutes les contraintes d'intégrité ; toujours dans un état valide.
 - **Isolation** : Plusieurs transactions peuvent s'exécuter simultanément sans interférer les unes avec les autres ; aucune interférence entre les transactions.
 - **Durabilité** : Une fois qu'une transaction est validée (commit), ses modifications sont définitivement enregistrées, même en cas de panne ou de crash du système ; la persistance est garantie.

Des données structurées au Big data

Données structurées

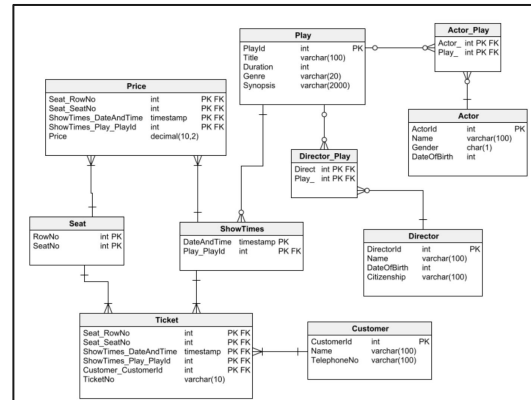
- **Limitations des SGBD relationnels :**
 - **Scalabilité :** optimisés uniquement pour une mise à l'échelle verticale (plus de ressources à 1 serveur) et non horizontalement (ajout de plusieurs servers)
=> Pas de possibilité de "sharding" (distribution de la donnée à travers les serveurs)



Des données structurées au Big data

Données structurées

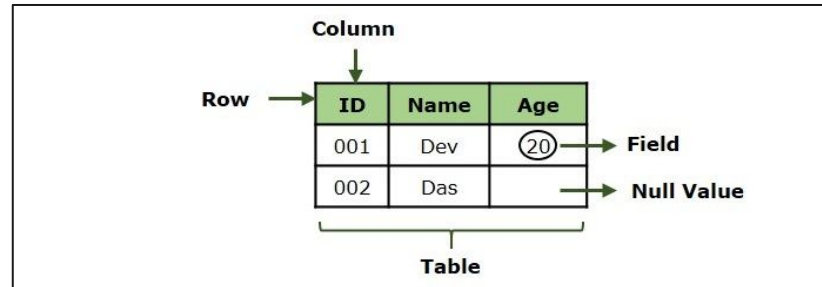
- Limitations des SGBD relationnels :
 - Rigidité des schémas : Ajouter ou modifier des colonnes requiert souvent des arrêts de service ou scripts complexes de migration
 - Contraintes : type, relations ...



Des données structurées au Big data

Données structurées

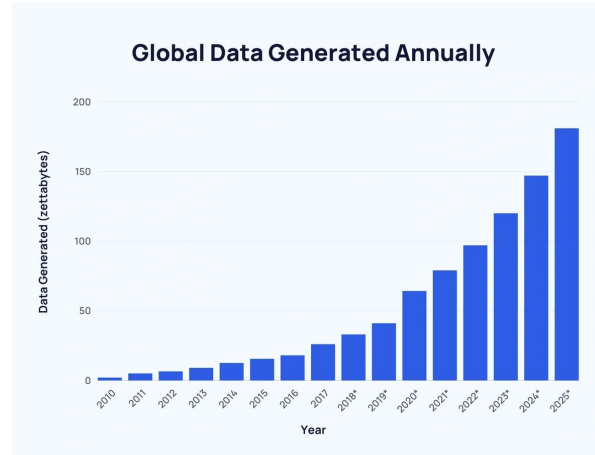
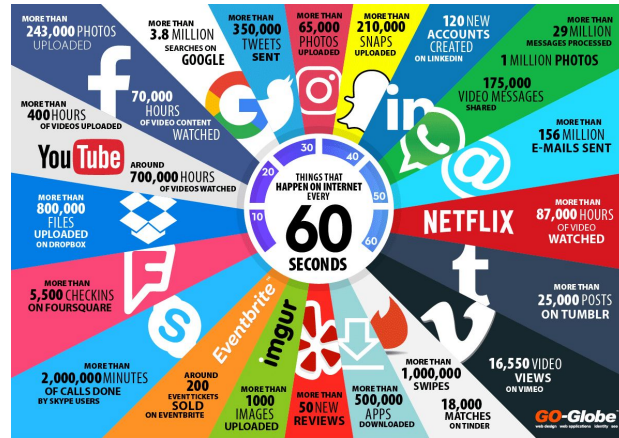
- **Limitations des SGBD relationnels :**
 - **Manque de flexibilité :** pas optimisés pour des données semi-structurées, non-structurées, hiérarchiques ou à haute vélocité (temps réel)
 - Images, Audio, Documents, Vidéos, JSON...



Des données structurées au Big data

L'ère du Big Data

- Explosion de la données issues :
 - des réseaux sociaux



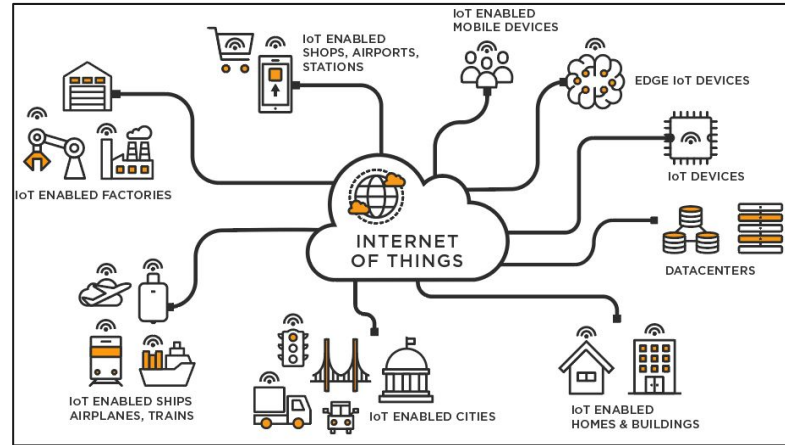
Multiples of bytes			
SI decimal prefixes		Binary usage	IEC binary prefixes
Name (Symbol)	Value		Name (Symbol)
kilobyte (kB)	10 ³	2 ¹⁰	kibibyte (KiB)
megabyte (MB)	10 ⁶	2 ²⁰	mebibyte (MiB)
gigabyte (GB)	10 ⁹	2 ³⁰	gibibyte (GiB)
terabyte (TB)	10 ¹²	2 ⁴⁰	tebibyte (TiB)
petabyte (PB)	10 ¹⁵	2 ⁵⁰	pebibyte (PiB)
exabyte (EB)	10 ¹⁸	2 ⁶⁰	exbibyte (EiB)
zettabyte (ZB)	10 ²¹	2 ⁷⁰	zebibyte (ZiB)
yottabyte (YB)	10 ²⁴	2 ⁸⁰	yobibyte (YiB)

See also: Multiples of bits • Orders of magnitude of data

Des données structurées au Big data

L'ère du Big Data

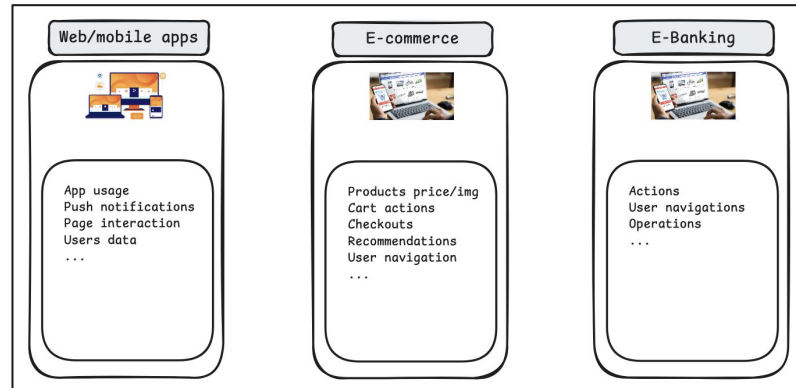
- Explosion de la données issues :
 - des IoT



Des données structurées au Big data

L'ère du Big Data

- Explosion de la données issues :
 - des logs and clickstreams



Require new models for storage & processing : real time, various formats, ...



Des données structurées au Big data

Notion de Big Data

Le **Big Data** : “désigne l'ensemble des **méthodes, architectures et technologies** permettant de **collecter, stocker, traiter et analyser** des **volumes très importants de données**, souvent **hétérogènes** et générées à **grande vitesse**, qui dépassent les capacités des systèmes informatiques traditionnels.”

- **4 composantes importantes :**
 - **des données massives** (volume) ;
 - **des infrastructures adaptées** (stockage et calcul distribués) ;
 - **des méthodes d'analyse** (statistiques, machine learning, intelligence artificielle) ;
 - **une finalité** : produire de la connaissance et aider à la décision.



Des données structurées au Big data

Caractéristiques des Big Data : les 5 V

- **Volume** – Masse gigantesque de données (terabytes aux petabytes)
- **Vélocité** – Données générées en temps réel
- **Variété** – Structurées, semi-structurées, non-structurées
- **Véracité** – Qualité & incertitudes dans la donnée
- **Valeur** – Transformer la données en insights



Des données structurées au Big data

Caractéristiques des Big Data : les 5 V

- **Volume**
 - Des volumes qui dépassent la mémoire d'un poste de travail classique
 - Pour les dataframes : le volume dépend du nombre de ligne, colonne et l'information stockée
 - Baromètre indicatif : « est-ce que ça tient dans Excel / la RAM ? »
 - **Attention** : le volume n'est pas un chiffre absolu, c'est un rapport à vos moyens. 5 Go tiennent sur un serveur mais peuvent faire planter un laptop.
 - Exemple : données de recensement de la population



Des données structurées au Big data

Caractéristiques des Big Data : les 5 V

- **Vélocité**
 - La vitesse à laquelle la donnée arrive et doit être traitée
 - Options de traitement :
 - Du batch (traitement différé)
 - Du temps réel (flux continu)
 - Exemples :
 - données de navigation sur un site ou une application mobile
 - transactions de paiement mobile



Des données structurées au Big data

Caractéristiques des Big Data : les 5 V

- **Variété**
 - Structurées (tables), semi-structurées (JSON, logs), non structurées (texte, images, GPS)
 - Nécessite d'autres modèles que la table relationnelle → NoSQL
 - Exemple :
 - un acte d'état civil (structuré) + mentions marginales (texte libre) + scan (image)
 - rapports transmis par les ministères (pdf)



Des données structurées au Big data

Caractéristiques des Big Data : les 5 V

- **Véracité**
 - Toutes les données massives ne sont pas de qualité
 - Contenus des réseaux sociaux/blogs/forums : avis propres des utilisateurs
 - Données générées par l'IA
 - Biais de couverture, doublons, valeurs manquantes, représentativité incertaine
 - Exemple :
 - les données de téléphonie couvrent les détenteurs de SIM, pas la population
 - pareil pour les comptes sur les réseaux sociaux



Des données structurées au Big data

Caractéristiques des Big Data : les 5 V

- **Valeur**
 - La donnée n'a de valeur que par l'usage qu'on en tire
 - Volume sans finalité = coût, pas richesse
 - Exemple :
 - colonne avec une valeur fixe dans une table
 - colonne dont la valeur peut être déduite d'une autre information, dans certains cas

Transition vers le NoSQL

Types de données modernes

Structured



Excel sheet

Semi-Structured



JSON



XML

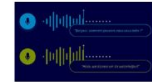


CSV

Unstructured



Images



Audio/Videos



Text



Transition vers le NoSQL

Notion de NoSQL

- NoSQL ⇔ Not Only SQL
- Des Base de données qui permettent de dépasser les limitations des SGBDR :
 - **Schémas flexibles** : peuvent stocker différents types de données
 - **Scalabilité horizontale** : Montée en échelle facilité par l'ajout de nouveaux servers
 - **Performance élevées** : bonne gestion de données massives, en temps réel, ...
 - **Haute disponibilité et tolérance au partitionnement** : distribuent la données entre servers



Le big data ... à l'ANSD

Quizz express : en groupes

- **Volume :**
 - Quel est le plus gros fichier que vous ayez eu à manipuler à l'ANSD, et est-ce que ça a coïncé ?
 - Un fichier de 200 Mo, est-ce du Big Data ?
- **Vélocité :**
 - Parmi les données que produit l'ANSD, lesquelles gagneraient à être disponibles en quasi temps réel ?
 - Vaut-il mieux une donnée parfaite dans six mois ou une donnée correcte demain ?
- **Variété :**
 - Lister les principaux types de données traitées par l'ANSD en donnant des exemples
 - Donnez un moyen/outil de stockage de l'ANSD pour des données qui n'entrent pas dans un tableau Excel
- **Véracité :**
 - Comment s'assurer de la correctitude de la donnée collectée ?
 - Une base de plusieurs millions de lignes est-elle forcément plus fiable qu'un échantillon de 1 000 personnes bien tiré ?
- **Valeur :**
 - Est-ce que les données de ticket de caisse (supermarché) peuvent être utiles aux statistiques produites par l'ANSD ?
 - Dans une table, si je garde une colonne timestamp et une colonne date, c'est un coût inutile ?



1.2

Environnement de travail &
Limites des outils classiques

Pré-requis et configuration de l'environnement

- **Prés-requis :**

- Les bases en statistiques et ML 
- Programmation Python  et connaissances en base de données 

- **Outils :**

- Python + Notebook lab 
 - VS Code  (ou temporairement Anaconda  ou Google colab 
[\(https://colab.research.google.com/\)](https://colab.research.google.com/)
- Compte Github 
 - Installation de git
- Installation de docker  : <https://docs.docker.com/desktop/setup/install/mac-install/>
- Miro : se connecter au tableau https://miro.com/app/board/uXjVH4b8L9M= (créer un compte si nécessaire)



Pré-requis et configuration de l'environnement

- Configuration de l'environnement sur *VS Code* :
 - Ouvrir **VS Code**
 - Aller sur le projet git et suivre les instructions :
 - <https://github.com/ababacaryoro/formation-bigdata/blob/main/INSTALLATION.md>



Limites des outils classiques

- **Objectif :**
 - Manipuler un jeu de données volumineux avec vos outils habituels
 - Constater et mesurer les limites : mémoire et vitesse
 - Repartir avec un diagnostic clair : pourquoi ça coince



Limites des outils classiques

- **Avant de commencer :**
 - Noter les caractéristiques de vos machines (total et niveau d'utilisation):
 - RAM : total et disponible
 - Processeur & nombre de coeurs
 - Stockage disponible
 - A mettre dans le board miro : https://miro.com/app/board/uXjVH4b8L9M=



Limites des outils classiques

TP 1.1 : en terrain connu

- Suivre les instructions dans [01-limites-du-poste/01_pandas_en_terrain_connu.ipynb](#) :
 - Charger un échantillon de 100 000 lignes
 - Explorer : dimensions, types, valeurs manquantes
 - Calculer : effectifs par région, âge moyen par sexe, taux d'activité
 - Notez le temps que ça prend — il servira de référence.



Limites des outils classiques

TP 1.2 : on monte le volume

- Suivre les instructions dans [01-limites-du-poste/02 le mur de la memoire.ipynb](#) :
 - Observez pendant le chargement (gestionnaire des tâches) :
 - le temps qui passe
 - la mémoire qui grimpe
 - Relancez les mêmes calculs qu'en partie 1
 - Consigne : notez chaque temps dans le tableau de mesures



Limites des outils classiques

TP 1.3 : mesurer la douleur du PC

- Suivre les instructions dans [01-limites-du-poste/03 echelle de la douleur.ipynb](#) :
 - Complétez le tableau :
 - opération $\times$ taille $\rightarrow$ temps mesuré
 - Lecture / filtre / group by / tri / jointure
 - Question :
 - quand la taille est $\times 20$:
 - le temps est $\times$ combien ?
 - la mémoire occupée est $\times$ combien ?



Limites des outils classiques

Diagnostic

- Mur n° 1 – la **mémoire** : pandas charge tout le fichier en **RAM**, d'un coup
- Mur n° 2 – le **mono-cœur** : un seul **processeur** travaille, les autres regardent
- Votre machine n'est pas mauvaise. L'outil n'est pas conçu pour ce volume.
- Ces deux murs structurent toutes les solutions de la semaine



Limites des outils classiques

Solutions ?

- Dépend de la situation

Volume de données	Solution	Quand
Jusqu'à quelques Go	Pandas optimisé	Machine locale
5–100 Go	Polars , DuckDB , Dask	Machine locale
100 Go–1 To	Dask distribué, Spark local	Petite infrastructure
>1 To	Spark Cluster	Véritable environnement Big Data



1.3

Concepts du **calcul distribué** &
Solutions pour données massives



Concepts du calcul distribué

- **Objectif :**
 - Comprendre pourquoi pandas atteint ses limites
 - Découvrir les leviers qui permettent d'aller plus loin
 - Situer les grandes solutions et leur zone de pertinence
 - Manipuler **Polars**, **DuckDB** et **Dask** sur votre fichier

Concepts du calcul distribué

Rappel : les 2 murs

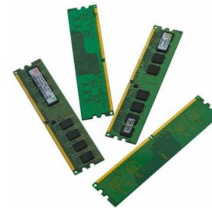
- Mur n° 1 – la mémoire :
 - tout le fichier doit tenir en RAM, plus la place pour travailler
- Mur n° 2 – le mono-cœur :
 - un seul processeur travaille, les autres attendent

Processeur



Exécute les instructions et calcule les données.
Rôle principal : Réfléchir, exécuter, traiter les calculs
Vitesse : Ultra-rapide mais traite peu de données à la fois.
Analogie : C'est un secrétaire qui résout les problèmes.

Mémoire RAM



La RAM stocke temporairement les données dont le processeur a besoin immédiatement.
Rôle principal : Stocker à court terme.
Vitesse : Très rapide, mais s'efface quand l'ordinateur s'éteint.
Analogie : C'est la taille du bureau où le secrétaire pose ses dossiers.



Concepts du calcul distribué

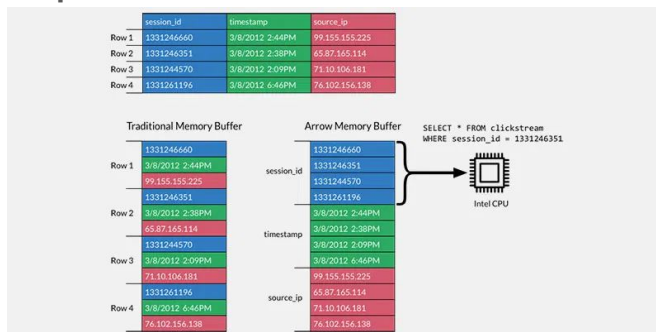
Leviers pour passer les murs

- **Colonnes** — ne lire que les colonnes utiles, dans un format compact
- **Lignes/Flux** — traiter par morceaux, ne jamais tout charger
- **Parallélisme** — occuper tous les cœurs de la machine
- **Paresse** — décrire le calcul, l'optimiser, puis l'exécuter

Concepts du calcul distribué

Leviers pour passer les murs

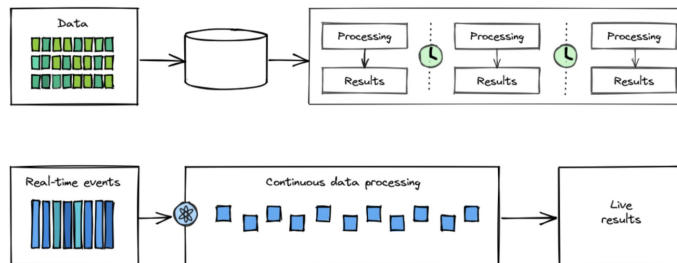
- **Levier 1 : le stockage en colonnes**
 - Les données d'analyse sont lues par colonne, pas par ligne
 - Stocker ensemble les valeurs d'une même colonne = lecture ciblée + compression pour les mêmes types de données
 - Format de référence : **Apache Arrow**



Concepts du calcul distribué

Leviers pour passer les murs

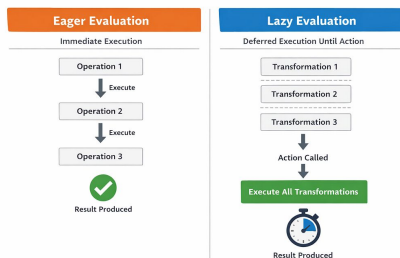
- **Levier 2 : le traitement en flux**
 - Lire un morceau, le traiter, l'oublier, passer au suivant
 - La mémoire nécessaire ne dépend plus de la taille du fichier
 - Fonctionne pour les filtres et les agrégations ; plus délicat pour les tris et jointures
 - C'est ce qui permet de traiter 100 Go avec 8 Go de RAM



Concepts du calcul distribué

Leviers pour passer les murs

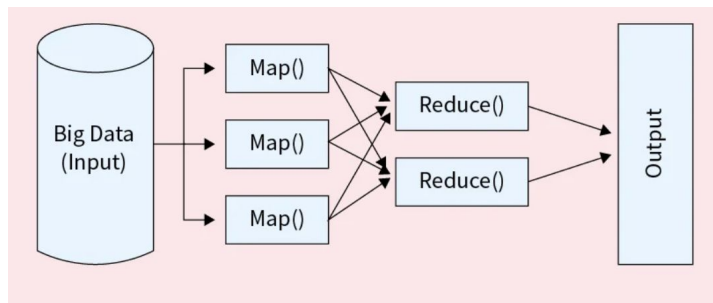
- **Levier 3 : l'exécution paresseuse**
 - Vous décrivez ce que vous voulez, sans l'exécuter
 - Le moteur construit un plan, l'optimise, puis l'exécute d'un coup
 - Optimisations typiques : ne lire que les colonnes utiles, filtrer au plus tôt, fusionner les étapes
 - Rien ne se calcule tant que vous ne demandez pas le résultat



Concepts du calcul distribué

Leviers pour passer les murs

- **Lever 4 : occuper tous les cœurs**
 - Votre poste a 4, 8 ou 16 cœurs ; pandas en utilise un
 - Une agrégation se découpe naturellement : chaque cœur traite une part, on additionne
 - C'est le principe **map / reduce** : découper, traiter en parallèle, recombinaison
 - Gain théorique proportionnel au nombre de cœurs





Concepts du calcul distribué

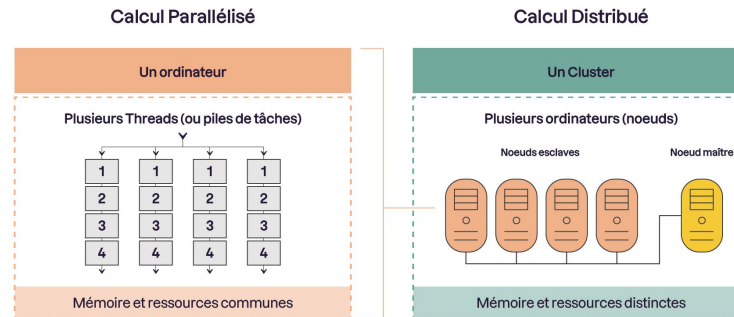
Leviers pour passer les murs

- **Notion de calcul distribué**
 - Ces 4 leviers sont des éléments d'optimisation essentiels pour le calcul distribué et/ou parallèle
 - **Calcul distribué** : consiste à répartir une tâche informatique sur plusieurs ordinateurs ou serveurs qui travaillent en parallèle et communiquent via un réseau.
 - Principe :
 - Découper un problème complexe → Distribuer les sous-tâches → Exécuter en parallèle → Fusionner les résultats

Concepts du calcul distribué

Leviers pour passer les murs

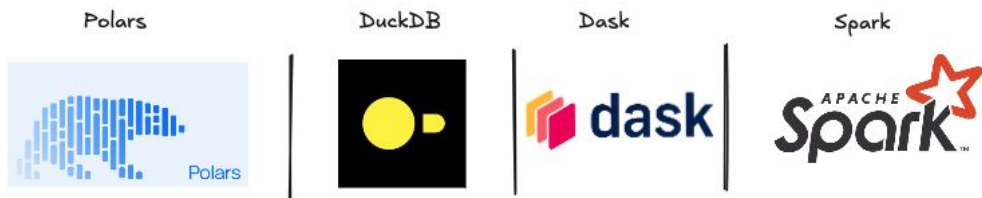
- **Calcul distribué Vs Calcul parallélisé**
 - **Calcul distribué** : Utilise plusieurs cœurs ou processeurs d'une même machine.
 - **Calcul parallélisé** : Utilise plusieurs machines (nœuds) reliées par un réseau.



Solutions pour données massives

Outils pour passer les murs

- 4 outils populaires :



- Comment choisir ?
 - dépend du volume
 - dépend de la machine
 - dépend des compétences dans l'équipe



Solutions pour données massives

Outils pour passer les murs

- **Polars :**
 - Écrit en Rust, format **Arrow** en interne
 - **Multi-cœur** par défaut, deux modes : immédiat et paresseux (*lazy* ou *eager* mode)
 - Moteur de flux (**stream**) pour dépasser la mémoire disponible
 - Syntaxe par expressions, proche de dplyr — un atout pour les utilisateurs de R
 - **Zone de pertinence** : machine unique, jusqu'à quelques **dizaines de Go**

Site : <https://pola.rs/>



Solutions pour données massives

Outils pour passer les murs

- **Polars :**

Créer un dataframe

```
import polars as pl

df = pl.DataFrame({
    "nom": ["Alice", "Bob", "Claire"],
    "age": [28, 35, 22],
    "ville": ["Paris", "Lyon", "Paris"]
})

print(df)
```

Opérations courantes

```
# Sélection de colonnes
df.select(["nom", "age"])

# Filtre
df.filter(pl.col("age") > 25)

# Ajout d'une colonne
df.with_columns(
    (pl.col("age") + 1).alias("age_an_prochain")
)
```

Chaîner les transformations

```
(
    df
    .filter(pl.col("ville") == "Paris")
    .group_by("ville")
    .agg(
        pl.col("age").mean().alias("age_moyen")
    )
)
```

- Les opérations s'expriment avec des expressions (`pl.col(...)`) plutôt qu'en manipulant directement les colonnes.

Solutions pour données massives

Outils pour passer les murs

- **Polars :**

Mode Lazy

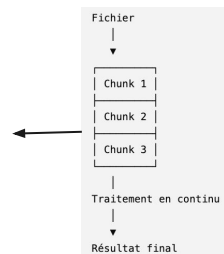
```
result = (  
    pl.scan_csv("ventes.csv")  
    .filter(pl.col("montant") > 100)  
    .group_by("client")  
    .agg(pl.sum("montant"))  
    .collect()  
)
```

- **scan_csv()** ne lit pas immédiatement le fichier.
- Les opérations sont enregistrées.
- L'exécution n'a lieu qu'au moment du **collect()**.
- Lecture uniquement des colonnes nécessaires
- Filtrage appliqué le plus tôt possible

Exécution par flux (stream)

```
result = (  
    pl.scan_parquet("logs.parquet")  
    .filter(pl.col("status") == 500)  
    .group_by("service")  
    .agg(pl.len().alias("nb_erreurs"))  
    .collect(engine="streaming")  
)
```

- **collect(engine="streaming")** permet d'activer la lecture en morceaux.
- Toutes les opérations ne sont pas compatibles avec le streaming (certaines jointures, fenêtres ou tris globaux).



- **read_*()** → exécution immédiate (eager)
- **scan_*()** → exécution différée (lazy), à privilégier pour les gros traitements.



Solutions pour données massives

Outils pour passer les murs

- **Polars :**

Support SQL

```
df = pl.DataFrame(  
    {  
        "country": ["USA", "USA", "USA", "USA", "USA", "Netherlands"],  
        "city": [  
            "New York",  
            "Los Angeles",  
            "Chicago",  
            "Houston",  
            "Phoenix",  
            "Amsterdam",  
        ],  
        "population": [8399000, 3997000, 2705000, 2320000, 1680000, 900000],  
    }  
)  
  
ctx = pl.SQLContext(population=df, eager=True)  
print(ctx.execute("SELECT * FROM population"))
```



Solutions pour données massives

Outils pour passer les murs

- **Polars** : à retenir
 - Lecture uniquement des colonnes nécessaires
 - Filtrage appliqué le plus tôt possible
 - Exécution immédiate (eager) ou différée (lazy) en fonction de l'opération
 - Peut traiter les données en blocs (stream) => analyse aisément des fichiers de plusieurs dizaines de Go, dépassant la RAM



Solutions pour données massives

Outils pour passer les murs

- **DuckDB :**
 - Une **base de données analytique** dans votre processus : pas de serveur, pas d'installation
 - **SQL** complet, exécution **vectorisée** et **multi-cœur**
 - **Interroge directement** un fichier CSV, Parquet ..., **sans les charger**
 - Déborde sur le **disque** quand la **mémoire** ne suffit pas
 - **Zone de pertinence** : analyse SQL, mêmes ordres de grandeur que Polars

Site : <https://duckdb.org/>



Solutions pour données massives

Outils pour passer les murs

- **DuckDB :**

Lire un fichier

```
import duckdb

con = duckdb.connect()

df = con.sql("""
SELECT *
FROM './00_data/individus.csv'
""").df()

print(df.head())
```

Lire et faire des opérations

```
import duckdb

con = duckdb.connect()

result = con.sql("""
SELECT *
FROM 'ventes.parquet'
WHERE montant > 100
LIMIT 10
""").df()

print(result)
```

Traiter plusieurs fichiers sans
charger en mémoire

```
duckdb.sql("""
SELECT
    customer_id,
    SUM(amount)
FROM 'transactions/*.parquet'
GROUP BY customer_id
""")
```

- DuckDB manipule directement les fichiers de données comme s'il s'agissait de tables.
- Peut traiter des jeux de données plus volumineux que la RAM



Solutions pour données massives

Outils pour passer les murs

- **DuckDB :**

Connection à une BD

```
import duckdb

con = duckdb.connect()

con.execute("INSTALL postgres")
con.execute("LOAD postgres")

con.execute("""
ATTACH 'dbname=ventes
        host=localhost
        user=postgres
        password=secret'
AS pg (TYPE postgres)
""")

df = con.sql("""
SELECT *
FROM pg.public.clients
WHERE pays = 'France'
""").pl()
```



Solutions pour données massives

Outils pour passer les murs

- **DuckDB** : à retenir
 - Lecture uniquement des colonnes utilisées (Projection Pushdown)
 - Filtrage au plus tôt (Predicate Pushdown)
 - Exécution vectorisée
 - Utilisation de tous les cœurs CPU disponibles
 - Traite des données qui dépassent les limites de la mémoire RAM



Solutions pour données massives

Outils pour passer les murs

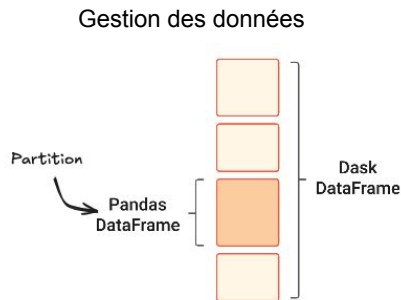
- **Dask :**
 - Bibliothèque Python pour le calcul **parallèle** et **distribué**
 - Fonctionne sur une machine ou sur un cluster
 - **Découpe** une **grande table** en **N tables plus petites** : les **partitions**
 - Construit un **graphe de tâches**, exécuté **parelleusement** sur appel de **.compute()**
 - API délibérément **proche de pandas** : le code change peu
 - **Zone de pertinence** : garder pandas à grande échelle, coordonner plusieurs machines

Site : <https://www.dask.org/>

Solutions pour données massives

Outils pour passer les murs

- **Dask :**

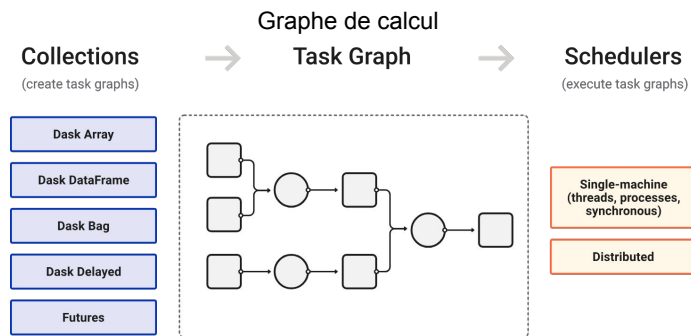


- Les données sont découpées en **partitions** et référencées dans un graphe de calcul.
- Les partitions sont chargées et calculées uniquement lorsqu'elles sont nécessaires (*lazy loading*).

Solutions pour données massives

Outils pour passer les murs

- **Dask :**



- Le graphe de calcul permet d'optimiser l'ordonnancement des calculs.
- Avant **.compute()**, aucun calcul n'est réellement exécuté

Solutions pour données massives

Outils pour passer les murs

- **Dask :**

Créer un dataframe

```
import pandas as pd
import dask.dataframe as dd

# Dask s'appuie sur un DataFrame Pandas
pdf = pd.DataFrame({
    "nom": ["Alice", "Bob", "Claire"],
    "age": [25, 42, 31],
    "ville": ["Paris", "Lyon", "Paris"]
})

# Création d'un DataFrame Dask (2 partitions)
df = dd.from_pandas(pdf, npartitions=2)

print(df)          # Affiche la structure
print(df.head())   # Affiche les premières lignes
```

Lire un fichier

```
import dask.dataframe as dd

df = dd.read_csv("../00_data/individus.csv")

print(df)          # Structure du DataFrame
print(df.head())   # Déclenche la lecture des premières lignes
```

Traiter plusieurs fichiers en mode lazy

```
import dask.dataframe as dd

resultat = (
    dd.read_parquet("../00_data/ventes/*.parquet")
    .query("montant >= 100")
    .groupby("region")
    .montant.sum()
    .sort_values(ascending=False)
    .compute()
)

print(resultat)
```

- Plusieurs optimisations : Projection Pushdown (colonnes), Filter/Predicate Pushdown (lignes), Partition Pruning (partitions), Avoiding Shuffles, Automatically resizing partitions



Solutions pour données massives

Outils pour passer les murs

- **Dask :**

Connection à une BD

```
import dask.dataframe as dd
from sqlalchemy import create_engine

url = "postgresql://user:password@localhost:5432/ma_base"

engine = create_engine(url)

df = dd.read_sql_table(
    table_name="Person",
    con=url,
    index_col="PersonID"
)

print(df.head())
```

- *index_col* est obligatoire. Dask partitionne la lecture à partir de cette colonne afin de paralléliser les requêtes SQL.



Solutions pour données massives

Outils pour passer les murs

- **Dask** : à retenir
 - API proche de Pandas
 - Calcul Lazy basé sur un graphe de tâches ;
 - Très adapté aux répertoires contenant de nombreux fichiers
 - Exécution parallèle sur une machine ou un cluster ;
 - Idéal lorsque les données dépassent la mémoire ou nécessitent une forte parallélisation.



Solutions pour données massives

Outils pour passer les murs

- **Apache Spark :**
 - La solution standard **Big Data** du **traitement** réellement **distribué**, sur **JVM**
 - Conçu pour exécuter des **traitements** sur **plusieurs machines**
 - **Tolérance aux pannes**, écosystème mature, **SQL** et flux **temps réel**
 - Coût d'entrée : **cluster**, configuration, exploitation
 - **Zone de pertinence** : au-delà de ce qu'une seule machine peut faire

Site : <https://spark.apache.org/>



Solutions pour données massives

Outils pour passer les murs

- **Apache Spark :**

Lecture de données

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.getOrCreate()

df = spark.read.parquet("../00_data/ventes")

result = (
    df.filter(df.montant >= 100)
      .groupBy("region")
      .sum("montant")
)

result.show()
```

- L'API est proche de Pandas, Dask et Polars.
- Sera détaillé plus tard



Solutions pour données massives

Outils pour passer les murs

- **Apache Spark** : à retenir
 - Adapté quand :
 - les données dépassent largement la capacité d'une seule machine ;
 - plusieurs utilisateurs partagent une plateforme de calcul ;
 - les traitements doivent être distribués sur un cluster.
 - Peut être lancé sur 1 seule machine, mais perd sa pertinence



Solutions pour données massives

Outils pour passer les murs

- **Synthèse globale :**
 - Leviers pour gérer de grosse volumétries :
 - **colonnes, lignes, parallélisme**, évaluation différée (**lazy**)
 - un autre levier à voir plus tard : le **format** des données
 - Plusieurs outils disponibles pour prendre en compte ces leviers :
 - **Pandas optimisé, Polars, DuckDB, Dask, Spark**
 - Une machine correcte (32 ou 64 Go) peut parfois suffir avec ces outils sans cluster
 - Jusqu'à ~100 Go, les moteurs sur machine unique sont parfois plus rapides et moins coûteux qu'un cluster
 - Monter un cluster « parce que c'est du Big Data » coûte cher et ralentit



Solutions pour données massives

TP 2.1 : Polars

- Suivre les instructions dans [02-franchir-le-mur/01_polars.ipynb](#)



Solutions pour données massives

TP 2.2 : DuckDB

- Suivre les instructions dans [02-franchir-le-mur/02_duckdb.ipynb](#)



Solutions pour données massives

TP 2.3 : Dask

- Suivre les instructions dans [02-franchir-le-mur/03_dask.ipynb](#)



Solutions pour données massives

TP 2.4 : Comparaisons

- Suivre les instructions dans [02-franchir-le-mur/04_comparaison.ipynb](#)



1.4

Formats de données
massives



Formats de données massives

- **Objectif :**
 - Comprendre pourquoi le **CSV** coûte cher, à chaque lecture
 - Situer les **principaux formats de données** et leurs usages
 - Découvrir ce qu'apporte un format colonnaire : **Parquet**
 - Mesurer le **gain** sur les différents outils



Formats de données massives

Inconvénients du CSV

- Aucun **type** stocké : le moteur doit les deviner
- **Orienté lignes** : lire 2 colonnes oblige à **parcourir tout le fichier**
- Aucune **compression**, aucune **métadonnée**, aucune **statistique**
- Il ne sait pas dire « la colonne âge va de 0 à 999 »
- Donc il ne peut pas éviter de **lire ce dont on n'a pas besoin**



Formats de données massives

Divers formats

- Deux caractéristiques pour les formats de données :
 - **Orienté ligne** (échange et écriture) ou **colonne** (pour analyse et lecture)
 - **Texte** (lisible par un humain) ou **binaire** (compact et typé)
 - CSV, JSON, XML : **lignes + texte**
 - Parquet, ORC : **colonnes + binaire**
 - Avro : **lignes + binaire**

Formats de données massives

Divers formats

Orienté ligne (échange et écriture) ou **colonne** (pour analyse et lecture)

Orienté ligne

```
Enregistrement 1
├─ id
├─ nom
├─ âge
└─ ville

Enregistrement 2
├─ id
├─ nom
├─ âge
└─ ville
```

Les lignes sont stockées les unes après les autres.

Orienté colonne

```
id
1
2
3
4

nom
Alice
Bob
Claire
David

âge
25
42
31
28
```

Chaque colonne est stockée séparément.

Formats de données massives

Divers formats

- Les formats d'échange

CSV

```
2024-12-12;Central;Douglas;John;Television;67;51 100,00;500 266,00
2024-12-29;East;Douglas;Karen;Video Games;74;550,50;14 329,00
2024-01-15;Central;Timothy;David;Home Theater;46;5500,00;523 000,00
2024-02-01;Central;Douglas;John;Home Theater;87;5500,00;543 500,00
2024-02-18;East;Martha;Alexander;Home Theater;4;5500,00;52 000,00
2024-03-07;West;Timothy;Stephen;Home Theater;7;5500,00;57 500,00
2024-03-24;Central;Hermann;Luis;Video Games;50;550,50;52 925,00
2024-04-10;Central;Martha;Steven;Television;60;91 100,00;579 000,00
2024-04-27;East;Martha;Diana;Cell Phone;96;1225,00;121 000,00
2024-05-14;Central;Timothy;David;Television;53;91 100,00;563 494,00
2024-05-31;Central;Timothy;David;Home Theater;100;5500,00;540 000,00
2024-06-17;Central;Hermann;Shelli;Desk;5;1125,00;6025,00
2024-07-04;East;Martha;Alexander;Video Games;62;550,50;53 027,00
2024-07-21;Central;Hermann;Sigal;Video Games;55;150,00;63 217,50
2024-08-07;Central;Hermann;Shelli;Video Games;42;550,50;52 457,00
2024-08-24;West;Timothy;Stephen;Desk;3;1125,00;5375,00
2024-09-10;Central;Timothy;David;Television;7;51 100,00;50 300,00
2024-09-27;West;Timothy;Stephen;Cell Phone;76;1225,00;117 100,00
2024-10-14;West;Douglas;Michael;Home Theater;57;5500,00;520 500,00
2024-10-31;Central;Martha;Steven;Television;14;91 100,00;516 772,00
2024-11-17;Central;Hermann;Luis;Home Theater;11;5500,00;55 500,00
2024-12-04;Central;Hermann;Luis;Home Theater;104;5500,00;547 000,00
2024-12-21;Central;Martha;Steven;Home Theater;20;5500,00;514 000,00
```

Universel, lisible, sans type

JSON

```
{
  "first_name": "John",
  "last_name": "Doe",
  "age": 19,
  "is_student": true,
  "hobbies": ["hiking", "running", "cooking"],
  "address": {
    "street": "123 Main St",
    "unit": null,
    "city": "Anytown",
    "state": "NY",
    "zip": "12345"
  }
},
{
  "first_name": "Sarah",
  "last_name": "Doe",
  "age": 25,
  "is_student": false,
  "hobbies": ["drawing", "hiking", "video gaming"],
  "address": {
    "street": "456 Oak St",
    "unit": "apt",
    "city": "Oxnard",
    "state": "CA",
    "zip": "93050"
  }
},
{
  "first_name": "Anthony",
  "last_name": "Doe",
  "age": 40,
  "is_student": false,
  "hobbies": ["traveling", "photography", "hiking"],
  "address": {
    "street": "789 Elm St",
    "unit": "B01"
  }
}
```

Hiérarchique, souple, verbeux.
Naturel pour les données imbriquées

XML

```
<?xml version="1.0"?>
<Dataset xmlns="http://www.safe.com">
  <Building id="Surrey Head Office">
    <Address>"7445 132 St."</Address>
    <City>Surrey</City>
    <Province>BC</Province>
    <Country>Canada</Country>
  </Location>
  <Longitude>-122.860</Longitude>
  <Latitude>49.138</Latitude>
</Location>
<Reference>https://www.google.ca/maps/
3m114b114m513m411s0x5485dbd520cc
122.8574636?hl=en</Reference>
  <Room id="Admin_100">
    <Name>Reception</Name>
    <Category>Admin</Category>
    <Area units="m2">12</Area>
  </Room id="Admin_100_WallA">
    <material>wood</material>
  </Wall>
```

Hiérarchique avec schéma strict.
Très présent dans les échanges administratifs

Formats de données massives

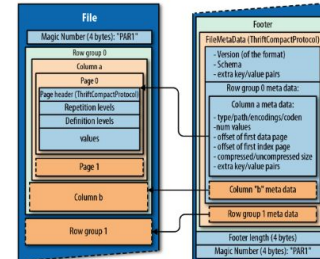
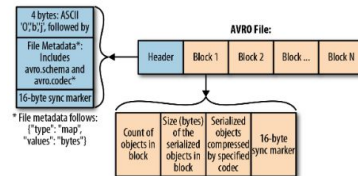
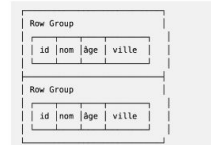
Divers formats

- Les formats de traitement

Avro



Parquet/ORC



Formats de données massives

Focus sur le format **Parquet**

- **Caractéristiques :**
 - Toutes les **valeurs d'une colonne** sont **contiguës** sur le disque
 - Le **schéma** et les **types** sont inscrits dans le fichier
 - Les données sont **compressées** après encodage : snappy, zstd, gzip
 - Plus on compresse, plus le fichier est petit, plus il faut de calcul pour le relire
 - Possible de partitionner suivant des variables du dataset => **lecture plus ciblée**

data-lake > 01_bronze > BusinessCentral > bt_bc_account > year=2025 > month=10 > day=22

Authentication method: Access key ([Switch to Microsoft Entra user account](#))

Only show active objects

Showing all 2 items

☐	Name	Last modified	Access tier	Blob type	Size	Lease state	
☐	[.]						...
☐	20251022205843_bt_bc_account.parquet	22/10/2025, 22:59:01	Cool (Inferred)	Block blob	13.26 KiB	Available	...
☐	20251022224604_bt_bc_account.parquet	23/10/2025, 00:46:17	Cool (Inferred)	Block blob	13.26 KiB	Available	...



Formats de données massives

TP 3.1 : Comparaison CSV - Parquet

- Suivre les instructions dans [03-formats-parquet/01 csv vs parquet.ipynb](#)



Formats de données massives

TP 3.2 : Evaluation de performances

- Suivre les instructions dans [03-formats-parquet/02 bilan.ipynb](#)



Big data : Fondamentaux et calcul parallèle

Synthèse globale

- **Synthèse globale :**
 - Limites des postes de calcul classiques :
 - la **mémoire**
 - le **mono-cœur**
 - Approches pour gérer de grosses volumétries en local :
 - Optimisations sur le moteur de calcul :
 - **Pandas optimisé, Polars, DuckDB, Dask, Spark**
 - Optimisation sur le format des données :
 - **Parquet, Arrow, Avro, ORC, ...**



Part 2 :

Architectures,
containerisation et NoSQL



2.1

Architectures Big Data



Architectures Big Data

- **Objectif :**
 - Comprendre les différences entre Data Warehouse et Data Lake
 - Identifier les composants historiques de l'écosystème Hadoop
 - Comprendre les principes d'une architecture distribuée
 - Choisir une base NoSQL adaptée à un besoin métier



Architectures Big Data

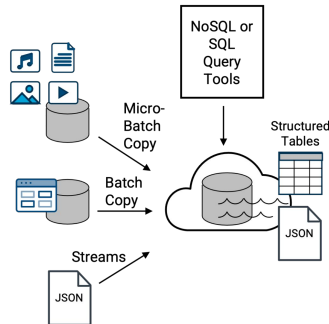
Problématiques des Big data

- Rappel des limites des SGBDR
 - Explosion du volume des données (TB → PB)
 - Diversité des formats
 - Structurées
 - Semi-structurées
 - Non structurées
 - Scalabilité horizontale plutôt que verticale
 - Les 5 V : volume, vitesse, variété, véracité, valeur
- 2 challenges à gérer :
 - Où et comment **stocker** la donnée?
 - Comment et avec quoi la **traiter** ?

Architectures Big Data

Problématique de stockage

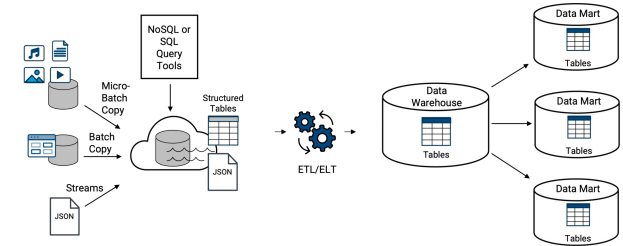
- **Data lake** =
 - “**Dépôt** centralisé, évolutif et sécurisé conçu pour **stocker, traiter et analyser** de **grands volumes** de données **structurées, semi-structurées** et **non structurées** dans leur **format natif**.”
 - Permet d'ingérer des données à **n'importe quelle vitesse** et dans **n'importe quel volume**



Architectures Big Data

Problématique de stockage

	Data Warehouse	Data Lake
Type de données	Données structurées	Tout type
Schéma	Schéma défini avant stockage	Schéma appliqué lors de la lecture
Qualité	Forte qualité des données	Données brutes
Usage	BI et reporting	IA, Data Science, exploration
Requêtage	SQL	SQL, Spark, Python, ...
Exemples	Snowflake, BigQuery, Amazon Redshift	HDFS, Amazon S3, Azure Data Lake Storage



Architectures Big Data

Problématique de stockage

- Amazon (AWS) S3

The screenshot displays the Amazon S3 console interface. On the left, the navigation pane shows the 'Amazon S3' service selected. The main content area shows the 'prmagfiles' bucket. Under the 'Objects' tab, three folders are listed: 'Issue1/', 'Issue2/', and 'Issue3/'. A blue arrow points from the 'Issue1/' folder to a detailed view of the 'Issue1/' folder. This view shows four objects:

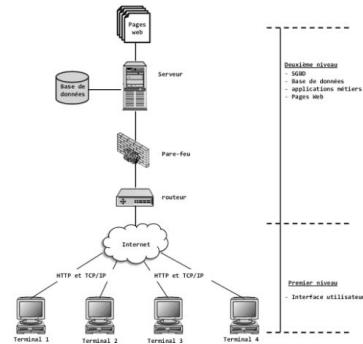
Name	Type	Last modified	Size	Storage class
ISSUE1.pdf	File	January 16, 2015, 10:13:00 UTC-08:00	116.2 KiB	Standard
Screenshot1.jpg	File	January 16, 2015, 10:08:34 UTC-08:00	296.9 KiB	Standard
Screenshot2.jpg	File	January 15, 2015, 11:09:25 UTC-08:00	8.0 KiB	Standard
Screenshot3.jpg	File	January 16, 2015, 15:55:22 UTC-08:00	37.5 KiB	Standard

Architectures Big Data

Problématique de traitement/calcul

- L'écosystème Hadoop (2006)

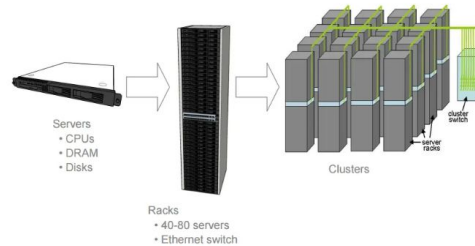
Approche traditionnelle



Stockage et le traitement des données centralisés dans un serveur.

- Serveurs spéciaux, très coûteux
- Peu de montée en charge

Approche introduite par HADOOP



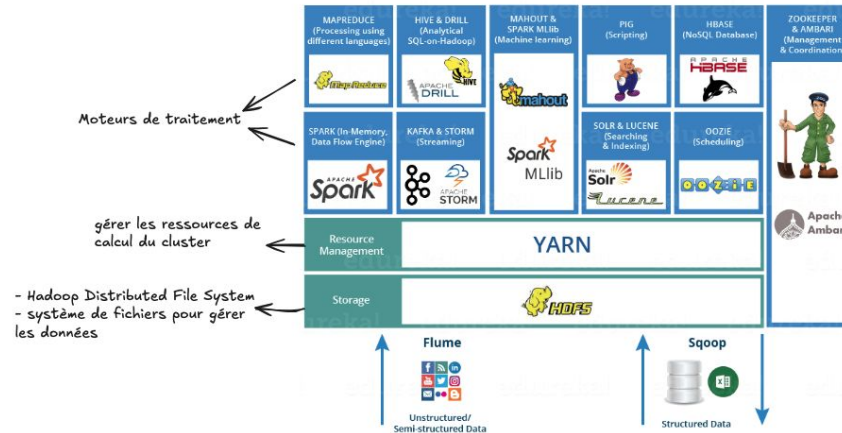
Distribuer le stockage des données et à paralléliser leur traitement sur plusieurs PC commodes organisés en cluster

- Distribuer les données
- Répartir les calculs et utiliser des serveurs standards

Architectures Big Data

Problématique de traitement/calcul

- **L'écosystème Hadoop**
 - 3 composants historiques : HDFS, MapReduce, YARN





Architectures Big Data

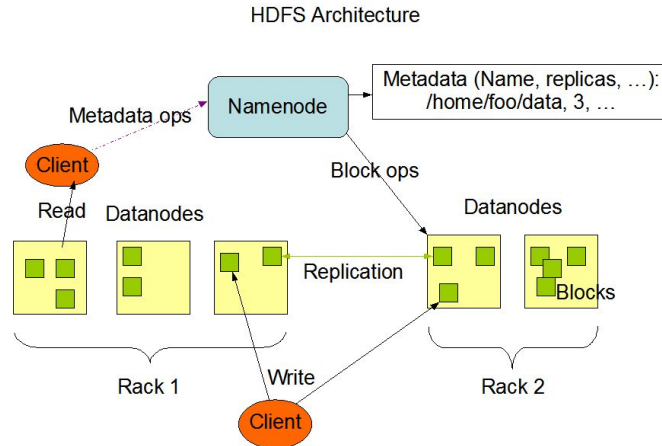
Problématique de traitement/calcul

- **L'écosystème Hadoop**
 - **HDFS**
 - **Système de gestion de fichiers** distribué : stockage principal d'Apache Hadoop
 - **Fonctionnement** :
 - **Découpage** des gros fichiers en **morceaux** répartis sur les machines du réseau
 - **Réplication** : chaque bloc est copié plusieurs fois sur des serveurs différents pour ne rien perdre si une machine s'arrête
 - **Architecture Maître/Esclave** : Un **nœud principal (NameNode)** gère l'organisation des fichiers, tandis que les **nœuds secondaires (DataNodes)** stockent les blocs de données
 - **Idée fondatrice** : déplacer le calcul vers la donnée, pas l'inverse

Architectures Big Data

Problématique de traitement/calcul

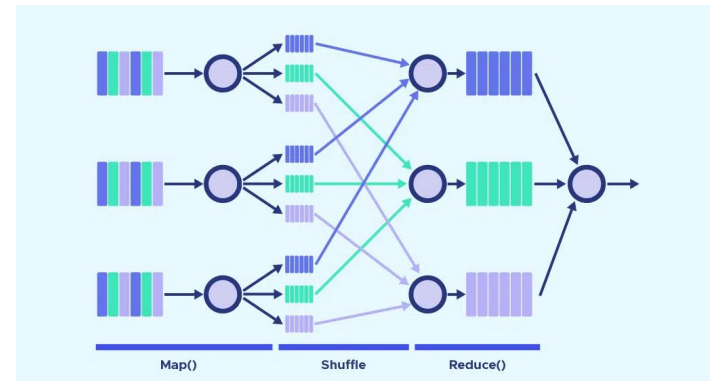
- L'écosystème Hadoop
 - HDFS



Architectures Big Data

Problématique de traitement/calcul

- **L'écosystème Hadoop**
 - **MapReduce**
 - C'est le **moteur de traitement distribué** d'Hadoop
 - Principe : découper le calcul en =>
 - une phase locale (**Map**) par server
 - un regroupement (**Shuffle**)
 - une agrégation (**Reduce**)
 - Chaque étape écrit sur le disque :
 - robuste, mais lent



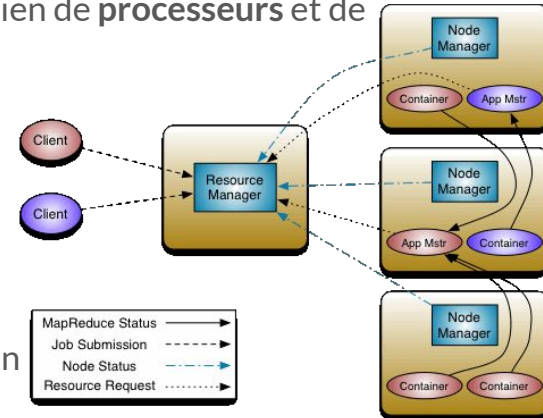
Architectures Big Data

Problématique de traitement/calcul

- **L'écosystème Hadoop**

- **YARN**

- Le **gestionnaire de ressources** : décide de qui obtient combien de **processeurs** et de **mémoire** sur le cluster
 - **Composants** :
 - **ResourceManager** : gère l'allocation des ressources
 - **NodeManager** :
 - surveille les ressources locales
 - lance les tâches demandées
 - **ApplicationMaster** :
 - pour chaque application, coordonne l'exécution



Architectures Big Data

Problématique de traitement/calcul

- **L'héritage d'Hadoop**
 - **MapReduce** : remplacé par **Spark**, qui travaille en mémoire
 - **HDFS** : largement supplanté par le stockage objet (**S3**, **Azure Blob Storage** et équivalents)
 - **YARN** : concurrencé par **Kubernetes**
 - Ce qui demeure : les concepts — blocs, réplication, calcul déporté, map/reduce





Architectures Big Data

Principes des architectures distribuées

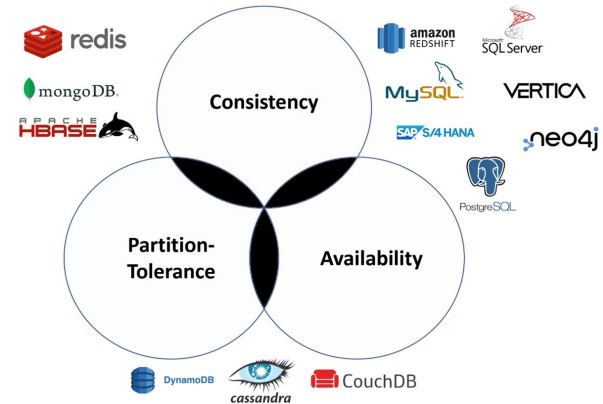
- **Le théorème CAP :**
 - Dans tout système distribué, il n'est pas possible de garantir simultanément ces **3 propriétés** :
 - **Cohérence (Consistency) :**
 - Tous les serveurs voient les mêmes données au même moment
 - **Disponibilité (Availability) :**
 - Chaque requête reçoit une réponse : le système reste toujours accessible et répond
 - **Tolérance au partitionnement (Partition Tolerance) :**
 - le système continue de fonctionner même si certaines parties ne peuvent plus communiquer entre elles (par exemple lorsqu'une liaison réseau est interrompue).
 - **Règle :**
 - On ne peut garantir que 2 propriétés sur les 3 à un instant donné. En cas de défaillance (panne de serveur ou problème réseau), il faut nécessairement en sacrifier une.

Architectures Big Data

Principes des architectures distribuées

- Le théorème CAP

Choix	Exemple	Implications
CP (Consistency + Partition)	HBASE, MongoDB	Données toujours exactes, mais potentiellement indisponibles en cas de problèmes réseau.
AP (Availability + Partition)	Amazon DynamoDB, Cassandra	Toujours réactif, mais les données sont parfois peut-être obsolètes.
CA (Consistency + Availability)	Bases de données traditionnelles (MySQL, PostgreSQL)	Nécessite un réseau parfait – Tous les clients peuvent lire et écrire les mêmes données.

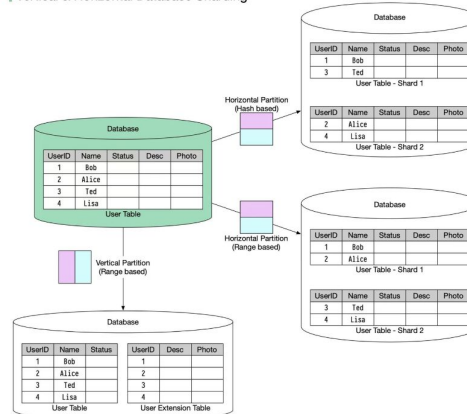


Architectures Big Data

Principes des architectures distribuées

- **Le partitionnement de la donnée :**
 - Données divisées en segments plus petits et plus faciles à gérer (partitions/shards) selon une stratégie.

Vertical & Horizontal Database Sharding



Key characteristics:

- Logical or physical separation of data based
- Each partition contains a subset of the complete dataset

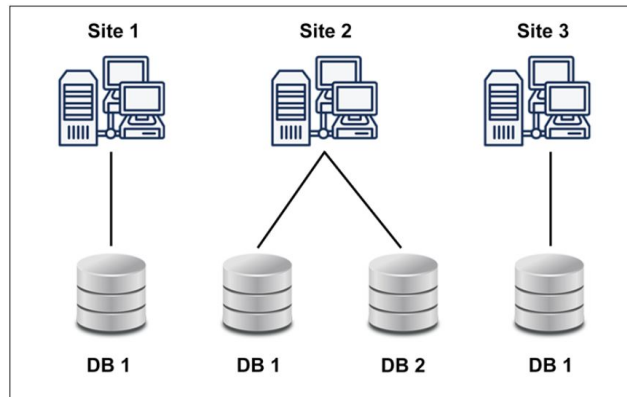
Purpose:

- Improved query performance
- More manageable data sizes
- Horizontal scalability

Architectures Big Data

Principes des architectures distribuées

- **La réplication de la donnée**
 - Les données sont réparties et copiées sur plusieurs machines physiques ou nœuds, généralement situés à des endroits différents.



Key characteristics:

- Geographic separation between nodes
- Replication of data across locations for availability/redundancy

Purpose:

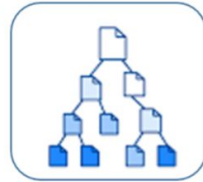
- High availability
- Disaster recovery
- Geographic performance optimization

Architectures Big Data

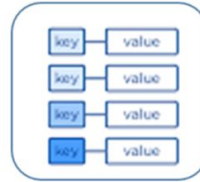
Les bases de données NoSQL

Plusieurs types :

Store semi-structured data as documents (typically JSON, XML...)
Use cases : user profiles, product catalogs
Ex. : MongoDB, DocumentDB

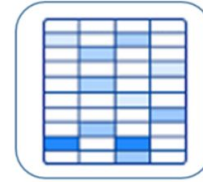


Document Store



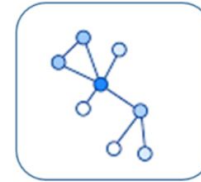
Key-Value Store

Simple data stores using unique keys to access values
Use cases : Caching, shopping carts
Ex. : Redis, DynamoDB



Wide-Column Store

Store data in column families that can be distributed across nodes
Use cases : event logging, IoT applications
Ex. : Cassandra, HBase



Graph Store

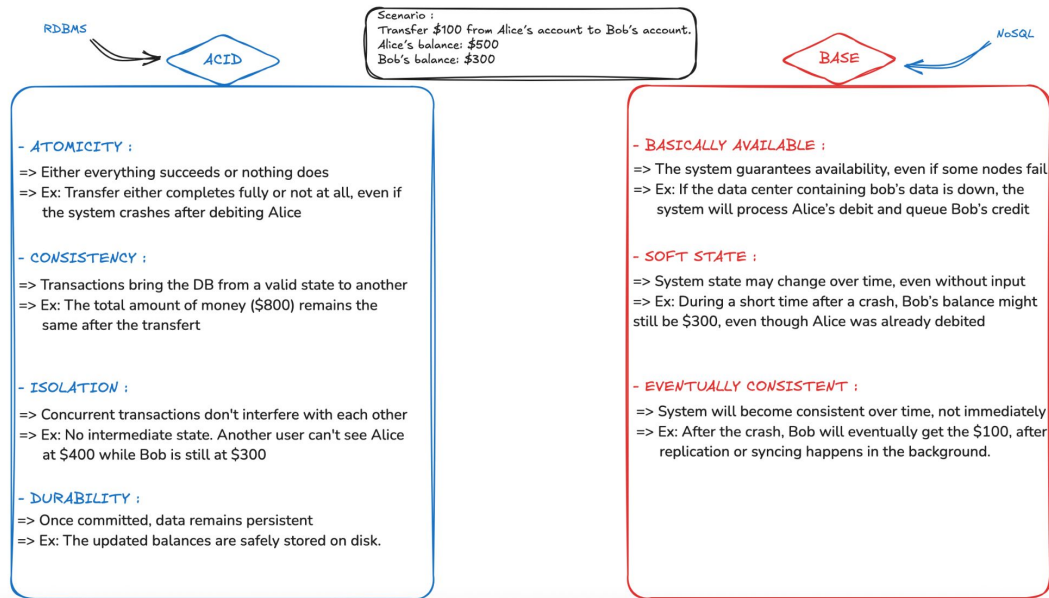
Specialized for highly connected data with explicit relationship modeling
Use cases : Social networks, recommendation engines
Ex. : Neo4j, JanusGraph

Architectures Big Data

Les bases de données NoSQL

Propriétés :

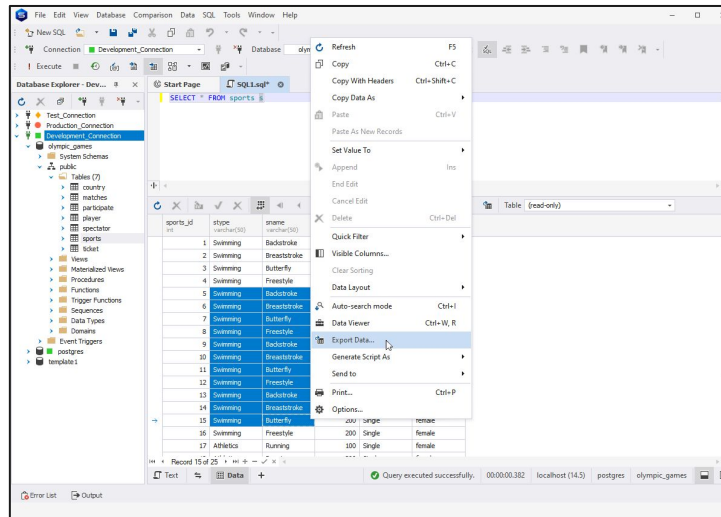
- BASE Vs ACID



Architectures Big Data

Les bases de données NoSQL

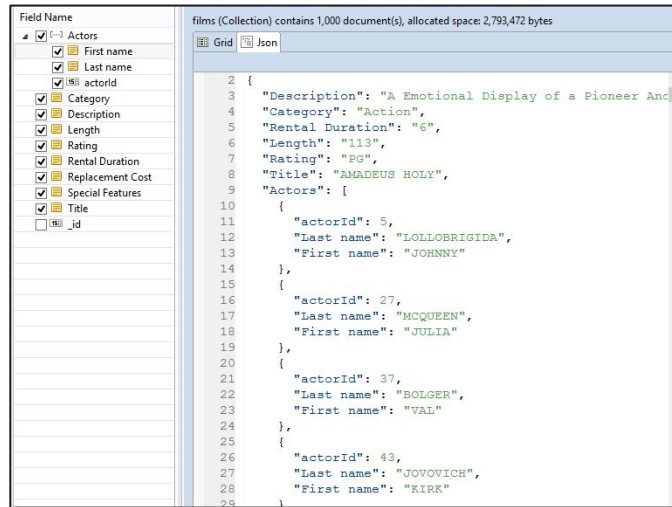
- BD relationnelles :



Architectures Big Data

Les bases de données NoSQL

- BD orientée Document:



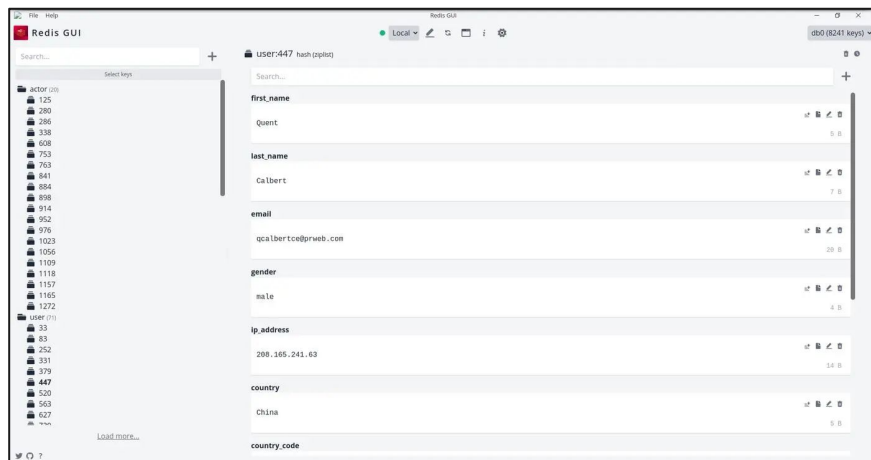
The screenshot displays the MongoDB web interface. On the left, the 'Field Name' list shows various fields for the 'Actors' collection, including 'First name', 'Last name', 'actorId', 'Category', 'Description', 'Length', 'Rating', 'Rental Duration', 'Replacement Cost', 'Special Features', 'Title', and '_id'. The 'films' collection is selected, and the 'Grid' tab is active. The main area shows a JSON document for a film, with the following fields and values:

```
{
  "Description": "A Emotional Display of a Pioneer And",
  "Category": "Action",
  "Rental Duration": "6",
  "Length": "113",
  "Rating": "PG",
  "Title": "AMADEUS HOLY",
  "Actors": [
    {
      "actorId": 5,
      "Last name": "LOLLOBRIGIDA",
      "First name": "JOHNNY"
    },
    {
      "actorId": 27,
      "Last name": "MCQUEEN",
      "First name": "JULIA"
    },
    {
      "actorId": 37,
      "Last name": "BOLGER",
      "First name": "VAL"
    },
    {
      "actorId": 43,
      "Last name": "JOVOVICH",
      "First name": "KIRK"
    }
  ]
}
```

Architectures Big Data

Les bases de données NoSQL

- BD orientée Clé-Valeur :



Architectures Big Data

Les bases de données NoSQL

- BD orientée Colonne :

Each cell has multiple versions, typically represented by the timestamp of when they were inserted into the table

Timestamp1 Timestamp2

Row Key	Column Family - Personal			Column Family - Office	
	Name	Residence Phone	Phone	Phone	Address
00001	John	415-111-1234	415-212-5544		1021 Market St
00002	Paul	408-432-9922	415-212-5544		1021 Market St
00003	Ron	415-993-2124	415-212-5544		1021 Market St
00004	Rob	815-243-9988	408-998-4322		4455 Bird Ave
00005	Carly	206-221-9123	408-998-4325		4455 Bird Ave
00006	Scott	818-231-2566	650-443-2211		543 Dale Ave

The table is lexicographically sorted on the row keys

Cells

```
hbase(main):009:0> create 'guru99', {NAME=>'e', VERSIONS=>6}
0 row(s) in 1.3790 seconds

=> Hbase::Table - guru99
hbase(main):010:0> put 'guru99', 'r1', 'e:c1', 'value', 10
0 row(s) in 0.0880 seconds

hbase(main):011:0> put 'guru99', 'r1', 'e:c1', 'value', 12
0 row(s) in 0.0090 seconds

hbase(main):012:0> put 'guru99', 'r1', 'e:c1', 'value', 14
0 row(s) in 0.0110 seconds

hbase(main):013:0> delete 'guru99', 'r1', 'e:c1', 11
0 row(s) in 0.0270 seconds
```

Creating Table guru99 and placing values in guru99

```
hbase(main):010:0> get 'guru99','r1', 'c1'
COLUMN
c1:
1 row(s) in 0.0440 seconds
```

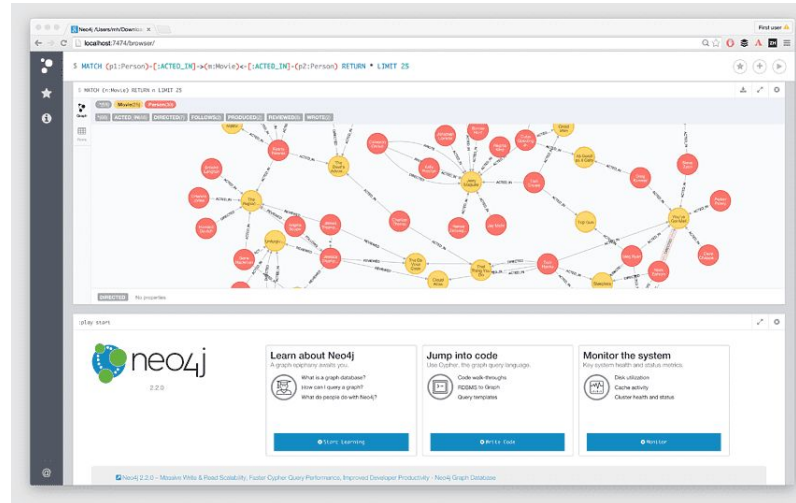
CELL
timestamp=30, value=value

getting records from 'guru99' using get

Architectures Big Data

Les bases de données NoSQL

- BD orienté Graphe :





2.2

La conteneurisation avec Docker



La conteneurisation avec Docker

- **Objectif :**
 - Comprendre ce qu'est un conteneur
 - Savoir démarrer, arrêter, inspecter et dépanner un service
 - Faire dialoguer deux conteneurs entre eux
 - Distinguer ce qui est jetable de ce qui persiste
 - Ne plus avoir de problèmes de compatibilité/versions



La conteneurisation avec Docker

Problématique

- Pour qu'une application de traitement de données marche, il faut :
 - Des librairies avec des versions spécifiques : **Pandas, Python, Java ...**
 - Des applications/frameworks installés sur plusieurs machines : **PostgreSQL, MongoDB, Spark, Kafka**
 - Bien gérer les **mise à jour** de bibliothèques pour ne pas casser ce qui marche ou éviter les **incompatibilités**
- **Solution :**
 - Emporter avec l'application tout ce dont elle a besoin = **conteneuriser** l'application



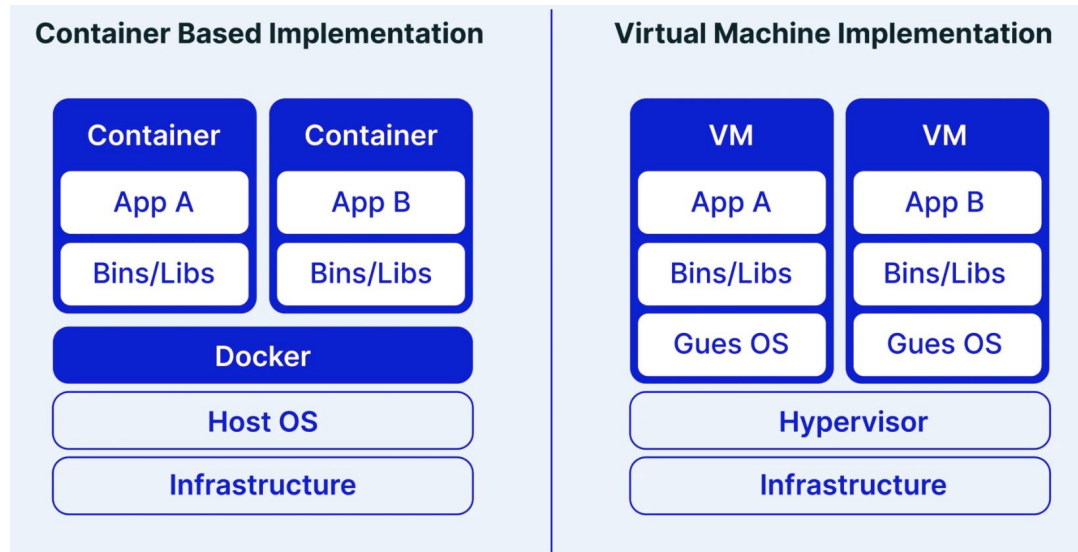
La conteneurisation avec Docker

La conteneurisation

- **Conteneurisation :**
 - Méthode qui consiste à **empaqueter** et **exécuter** des **applications** avec **toutes leurs dépendances** dans un **environnement léger, portable** et **isolé**, appelé conteneur.
- Différente de la **virtualisation** :
 - Créer une couche d'abstraction au-dessus du matériel (hardware), permettant à un **même ordinateur** physique d'être **divisé** en plusieurs **machines virtuelles**.
- **Un conteneur :**
 - Environnement **autonome**, comparable à une machine virtuelle, mais **sans la surcharge** d'un système d'exploitation complet. Il contient :
 - **L'application** (par exemple, une base de données)
 - Ses **dépendances** (bibliothèques, binaires, fichiers de configuration)
 - Tout le **nécessaire** pour exécuter l'application, quel que soit le système hôte.

La conteneurisation avec Docker

La conteneurisation





La conteneurisation avec Docker

La conteneurisation pour les données

Pourquoi conteneuriser dans le contexte des (bases de) données ?

- Portabilité, Déploiement facile et consistant
 - Les conteneurs marchent suivant différents environnements : Linux, Mac, Windows, plateformes cloud.
 - Les applications se comporteront de la même manière en local, en recette ou production...
- Développements et tests accélérés
 - Les développeurs peuvent tester plusieurs versions d'applications sans programme complexe.
 - Infrastructure as code: tout le setup des programmes sont définis dans un fichier de configuration.
 - Possible d'isoler l'environnement de tests facilement.

Remarques pour les applications en production :

- Les conteneurs sont **éphémères** : par défaut, les données sont perdues quand le conteneur est stoppé sans précautions.
- Il est donc nécessaire de toujours persister les données importantes



La conteneurisation avec Docker

La solution Docker



C'est quoi Docker?

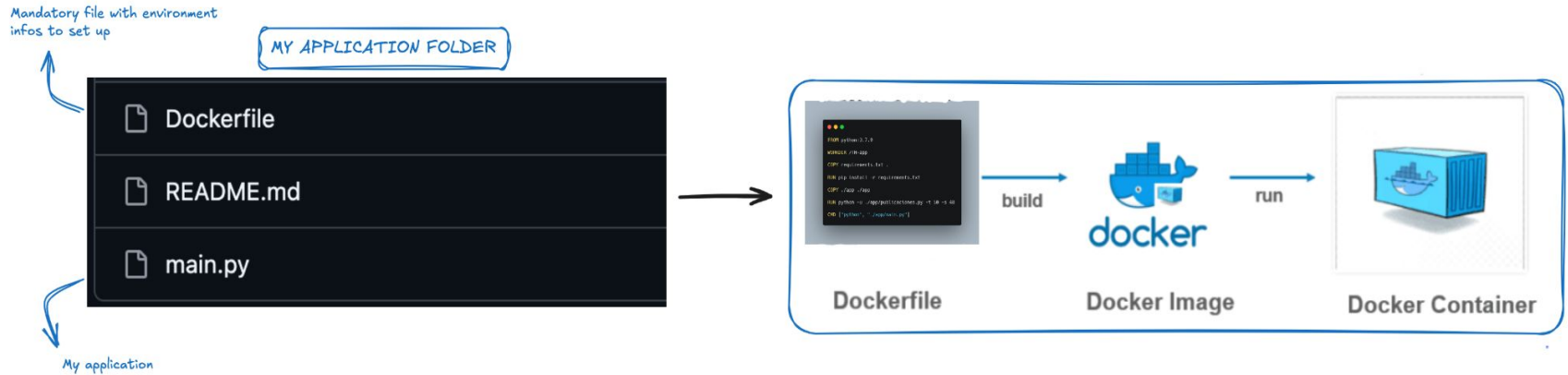
- Une plateforme open source qui permet d'automatiser le déploiement, la mise à l'échelle et la gestion d'applications légers installés dans des conteneurs.
- Un conteneur Docker package une application avec toutes ses dépendances (bibliothèques, fichiers de configuration, fichiers système) de sorte à pouvoir l'exécuter sur différents environnements.

Comment marche Docker?

- Un fichier **Dockerfile** décrit l'environnement et ses dépendances
- Une image Docker est construite à partir de ce fichier
- Cette image est exécutée dans un conteneur isolé, indépendant

La conteneurisation avec Docker

La solution Docker

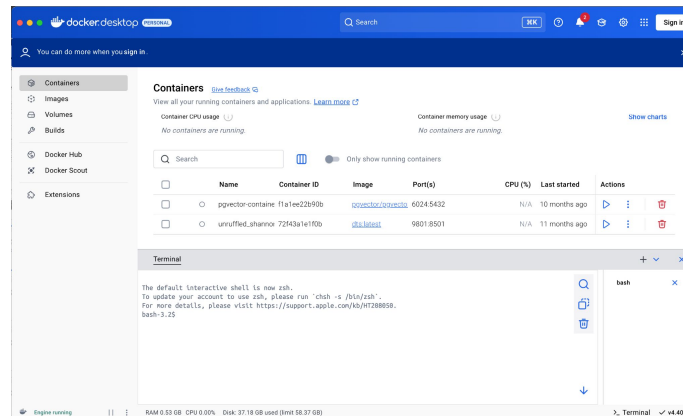


La conteneurisation avec Docker

Docker : prise en main

Version Desktop :

- Windows : <https://docs.docker.com/desktop/setup/install/windows-install/>
- Mac OS : <https://docs.docker.com/desktop/setup/install/mac-install/>

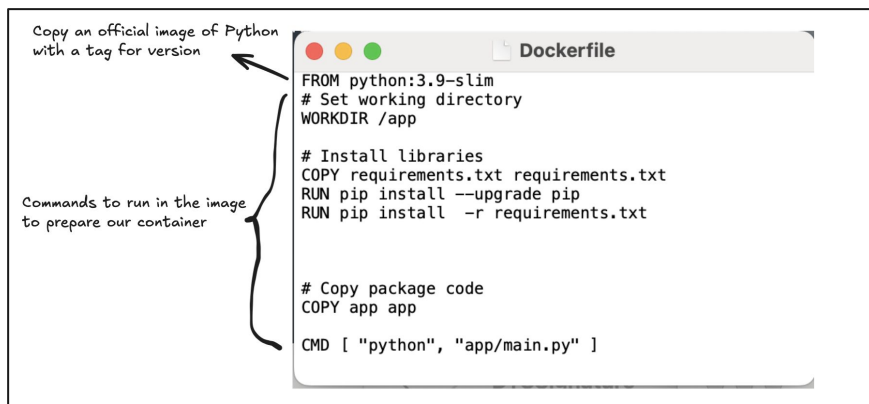


La conteneurisation avec Docker

Construire et lancer un conteneur

Supposons que l'on ait un programme python (*main.py*) que nous voulons exécuter dans un conteneur. Il y a 3 étapes à faire :

- > Etape 1 - Créer un **Dockerfile** dans le même répertoire



La conteneurisation avec Docker

Construire et lancer un conteneur

Supposons que l'on ait un programme python (*main.py*) que nous voulons exécuter dans un conteneur. Il y a 3 étapes à faire :

- > Etape 2 - Construire l'image Docker

Open a terminal in the directory with your Dockerfile and run

```
bash
```

```
docker build -t my-python-app .
```

names the image

current directory as context

La conteneurisation avec Docker

Construire et lancer un conteneur

Supposons que l'on ait un programme python (main.py) que nous voulons exécuter dans un conteneur. Il y a 3 étapes à faire :

- > Etape 3- Lancer l'image Docker dans un conteneur

```
bash
docker run -it --name my-container my-python-app
```

gives a name to the container

name of the image to run

NB : Si le conteneur lance uniquement un code, il s'arrêtera après exécution de ce code



La conteneurisation avec Docker

Construire et lancer un conteneur

Autres commandes Docker

- **Liste des conteneurs :**
 - `docker ps`
- **Vérifier les logs d'un conteneur :**
 - `docker logs my-container`
- **Arrêter un conteneur :**
 - `docker stop my-container`
- **Supprimer un conteneur :**
 - `docker rm my-container`
- **Copier une image docker depuis Docker hub :**
 - `docker pull THE_OFFICIAL_IMAGE_NAME`



La conteneurisation avec Docker

TP1 : Construire un conteneur

- **Application docker basique :**
 - Construire et lancer votre propre application docker qui exécute un programme python en vous basant sur ce contenu :
https://github.com/ababacaryoro/formation-bigdata/blob/main/04-docker/1-Docker-init/GUIDE_TP1.md

NB : N'hésitez pas à faire un tour sur le fichier de dépannage s'il y a une erreur :

- <https://github.com/ababacaryoro/formation-bigdata/blob/main/04-docker/DEPANNAGE.md>



La conteneurisation avec Docker

Docker Compose : les applications multi-conteneurs

- Lorsqu'une application doit exécuter **plusieurs services** pour des tâches différentes => plusieurs applications Docker.
- Au lieu d'exécuter des commandes ``docker run`` individuelles pour chaque service, **Docker Compose** vous permet de les définir dans un fichier nommé ``docker-compose.yml``.
- Exemple : une application web qui nécessite une base de données, un cache et peut-être un processus d'arrière-plan.
- Bénéfices :
 - **Configuration centralisée** pour tous les conteneurs
 - **Mise en place et arrêt** faciles d'une configuration multi-conteneur
 - **Mise en réseau** automatique entre les conteneurs
 - Prise en charge du montage de **volumes**, des **variables d'environnement** et du **mappage de ports**



La conteneurisation avec Docker

Docker Compose : les applications multi-conteneurs

- Dans un fichier ``docker-compose.yml``, on peut définir :
 - un **volume** nommé :
 - Espace géré par Docker, pour les données d'une base
 - Un **dossier** monté :
 - Vrai dossier de votre disque, accessible à travers le service associé
 - Un **réseau** commun :
 - Réseau à travers lequel chaque service peut être appelé par son nom
 - Un mappage des **ports** ("départ:arrivée") :
 - Un numéro pour le port exposé sur la machine local et un numéro pour le port sur le conteneur



La conteneurisation avec Docker

Docker Compose : les applications multi-conteneurs

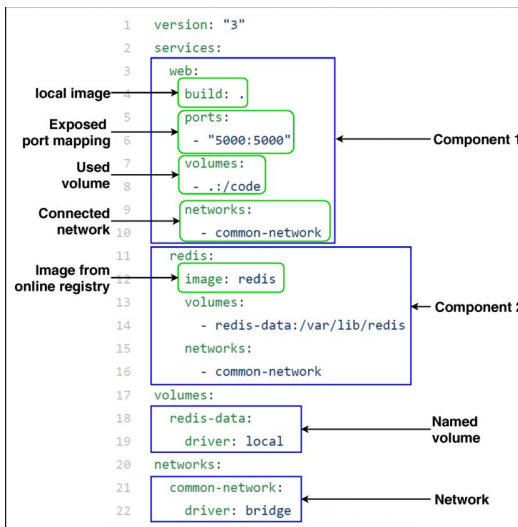
- Anatomie du fichier `docker-compose.yml` :
 - `services:` — un bloc par conteneur
 - `image:` ou `build:` d'où vient l'image
 - `ports:`, `volumes:`, `environment:` réglages pour les ports, les volumes et les variables d'environnement
 - `depends_on:` — l'ordre de démarrage/ les dépendances
 - `volumes:` en fin de fichier — les espaces de stockage déclarés
 - Les identifiants ne sont jamais dans ce fichier : ils viennent d'un `.env`

La conteneurisation avec Docker

Docker Compose : les applications multi-conteneurs

Étapes pour la création d'une application multi-conteneur

- Étape 1 - Créer le fichier *docker-compose.yml*



La conteneurisation avec Docker

Docker Compose : les applications multi-conteneurs

Étapes pour la création d'une application multi-conteneur

- **Étape 2 - Démarrer l'application**
 - `docker-compose up` ou `docker compose up` sur MacOS
- **Autres commandes :**
 - Arrêter l'application : `docker-compose down`
 - Créer les services : `docker-compose build`
 - Voir les logs des services : `docker-compose logs`
 - Lister les conteneurs : `docker-compose ps`

Containers [Give feedback](#)

View all your running containers and applications. [Learn more](#)

Container CPU usage 0.01% / 1200% (12 CPUs available) Container memory usage 254.93MB / 7.48GB

Search

☐ Only show running containers

☐	Name	Container ID	Image	Port(s)
☐	☐ pgvector-container	f1a1ee22890b	pgvector/pgvector:pg16	6024-5432
☒	☒ multi-container	-	-	-
☐	☒ db-1	5c66e6cb02c5	postgres:15	
☐	☒ notebook-1	d717d4ae006a	jupyter/scipy-notebook	8888:8888



La conteneurisation avec Docker

TP1 : Construire un conteneur

- **Application multi-conteneur :**
 - Construire et lancer une application multi-conteneur en vous basant sur ce contenu :
https://github.com/ababacaryoro/formation-bigdata/blob/main/04-docker/2-Multi-container/GUIDE_TP2.md

NB : N'hésitez pas à faire un tour sur le fichier de dépannage s'il y a une erreur :

- <https://github.com/ababacaryoro/formation-bigdata/blob/main/04-docker/DEPANNAGE.md>



La conteneurisation avec Docker

Conteneurisation avec Docker

À retenir :

- **Image** — la recette, figée, réutilisable. On la télécharge ou on la construit
- **Conteneur** — le plat préparé à partir de la recette. Il vit, puis on le jette
- **Volume** — le placard, qui survit au repas. Les données vont là-dedans
- **Réseau** — la salle à manger où plusieurs plats sont servis



2.3

Les Bases de données Big Data
orientées **Document** :
MongoDB



BD orientée Document : MongoDB

- **Objectif :**
 - Comprendre le modèle document et ce qu'il résout
 - Savoir arbitrer entre imbriquer et référencer
 - Interroger, agréger, indexer
 - Situer réplication et partitionnement : comment MongoDB passe à l'échelle



BD orientée Document : MongoDB

Concepts de base documentaire

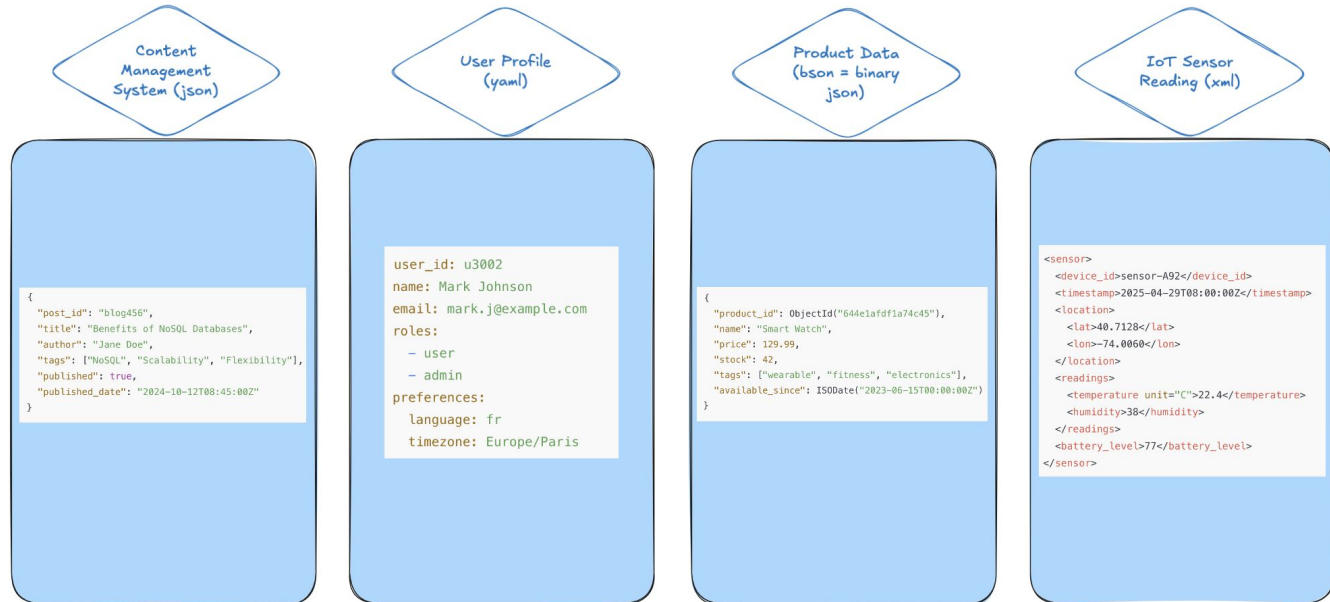
- Une **base documentaire** est un type de base de données NoSQL fait pour stocker, requêter et gérer des données semi-structurées en forme de documents, typiquement en format JSON, BSON, ou XML.
- Chaque **document** est :
 - Une **unité autonome** (comme une ligne en SQL)
 - Peut avoir plusieurs **champs imbriqués**, de type **scalaire** (textuel, numérique), **array**, or **objets imbriqués**
 - Les documents sont regroupés en **collections** (équivalentes aux tables en SQL)
- **Exemple de document :**

```
{
  "id": "user123",
  "name": "Alice",
  "email": "alice@example.com",
  "roles": ["admin", "editor"],
  "profile": {
    "age": 29,
    "location": "Paris"
  }
}
```

BD orientée Document : MongoDB

Concepts de base documentaire

Cas d'usage





BD orientée Document : MongoDB

Concepts de base documentaire

- **Caractéristiques :**
 - **Absence de schéma :** les structures des documents peuvent varier.
 - **Syntaxe de requêtage riche :** supporte les filtres et agrégations.
 - **Indexation :** possible d'indexer des champs à l'intérieur des documents pour des requêtes rapides.
 - **Scalability :** conçu pour une mise à l'échelle horizontale (via le sharding ou partitionnement)
- **Limitations :**
 - **Pas de contrôle strict du schéma :** les documents d'une même collection peuvent avoir des structures incohérentes.
 - **Duplication et redondance des données :** les données sont souvent dénormalisées (copiées entre les documents).
 - **Limitations des jointures :** la plupart des bases de données documentaires ne prennent pas en charge nativement les jointures comme SQL.



BD orientée Document : MongoDB

Concepts de base documentaire

BD relationnelle	MongoDB
Base	Base
Table	Collection
Ligne	Document
Colonne	Champ
Jointure	Imbrication (ou \$lookup)



BD orientée Document : MongoDB

MongoDB

- **MongoDB :**
 - Base de données NoSQL open source populaire, orientée documents
- **Caractéristiques clés :**
 - Stocke les données au format **BSON** (JSON binaire)
 - **Sans schéma** : structure de document flexible
 - Idéal pour les modèles de données semi-structurés ou évolutifs



BD orientée Document : MongoDB

MongoDB

- Structure des données :
 - Base de données → contient plusieurs **collections**
 - Collection → contient plusieurs **documents**
 - Document → un objet de type **JSON** (`{ "name": "Alice", "age": 30 }`)
- Types de données communs :

Data Type	Example	Notes
String	<code>"name": "Alice"</code>	Most commonly used data type
Integer	<code>"age": 30</code>	32-bit or 64-bit depending on the server
Double	<code>"price": 19.99</code>	For floating-point numbers
Boolean	<code>"active": true</code>	<code>true</code> or <code>false</code>
Date	<code>"created_at": ISODate(...)</code>	Stores dates with millisecond precision
ObjectId	<code>"_id": ObjectId(...)</code>	Unique identifier for each document
Array	<code>"tags": ["tech", "news"]</code>	List of values
Embedded Document	<code>"address": { "city": "Paris" }</code>	Document inside a document
Null	<code>"deleted_at": null</code>	Explicitly represents a missing field

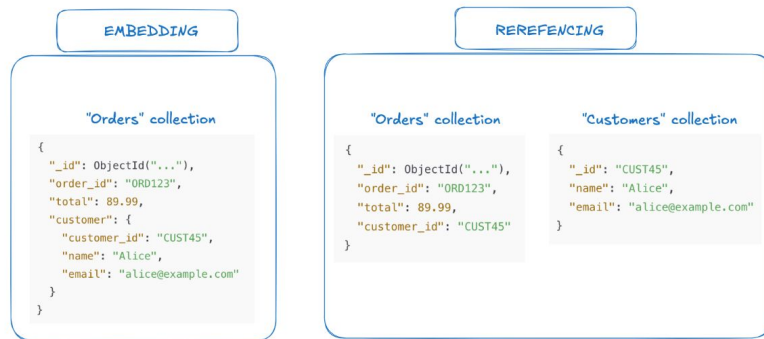
BD orientée Document : MongoDB

Modélisation d'un BD MongoDB

Pas de schéma requis, mais 2 options à arbitrer :

- **Imbriquer (embedding) :**
 - l'objet vit dans le document parent
- **Références (referencing) :**
 - le document ne porte qu'un identifiant vers un autre document

L'arbitrage se fait sur l'usage, pas sur l'esthétique



🧐 Decision Guide

Criteria	Embed	Reference
Data is always read together	✅ Yes	❌ No (requires join or extra query)
Data changes frequently	❌ No (duplication risk)	✅ Yes (update once)
One-to-few relationship	✅ Embed is preferred	❌ Reference adds overhead
One-to-many or many-to-many	❌ Embedding can bloat document	✅ Use referencing



BD orientée Document : MongoDB

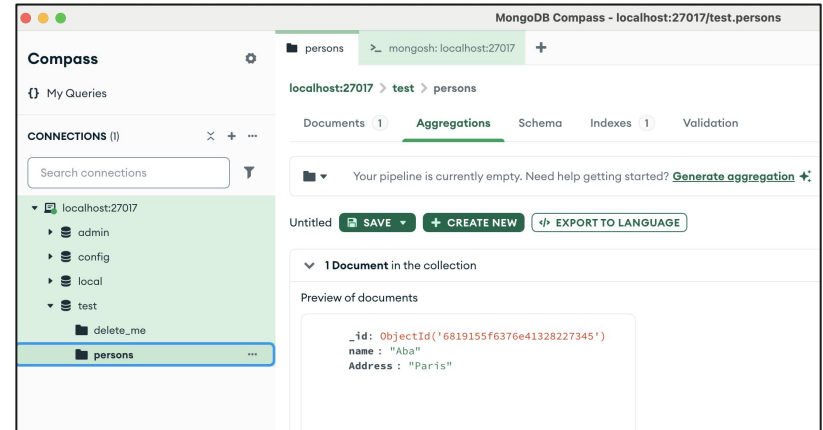
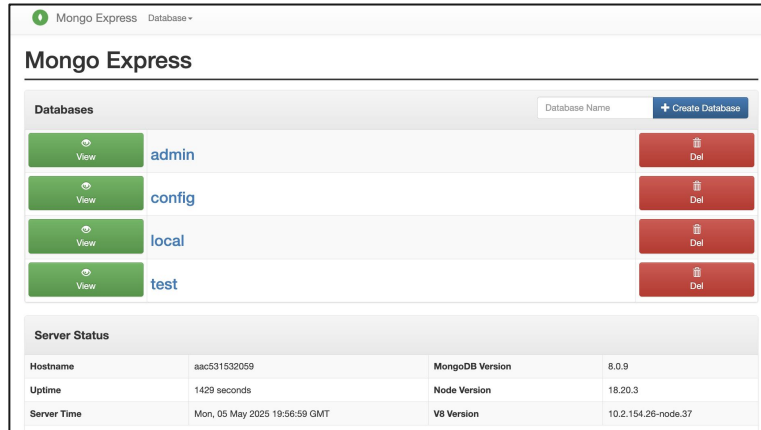
MongoDB : Installation et prise en main

- **Installation en local:**
 - Client Desktop : <https://www.mongodb.com/docs/manual/installation/>
- **Installation via Docker : install using docker compose**
 - Repo :
 - https://hub.docker.com/_/mongo
 - <https://www.mongodb.com/docs/manual/tutorial/install-mongodb-community-with-docker/>
 - Image MongoDB => `image: mongo:latest`
 - Interface utilisateur => **Mongo-express** `image: mongo-express:latest` ou **MongoDB Compass** (<https://www.mongodb.com/try/download/compass>)

BD orientée Document : MongoDB

MongoDB : Installation et prise en main

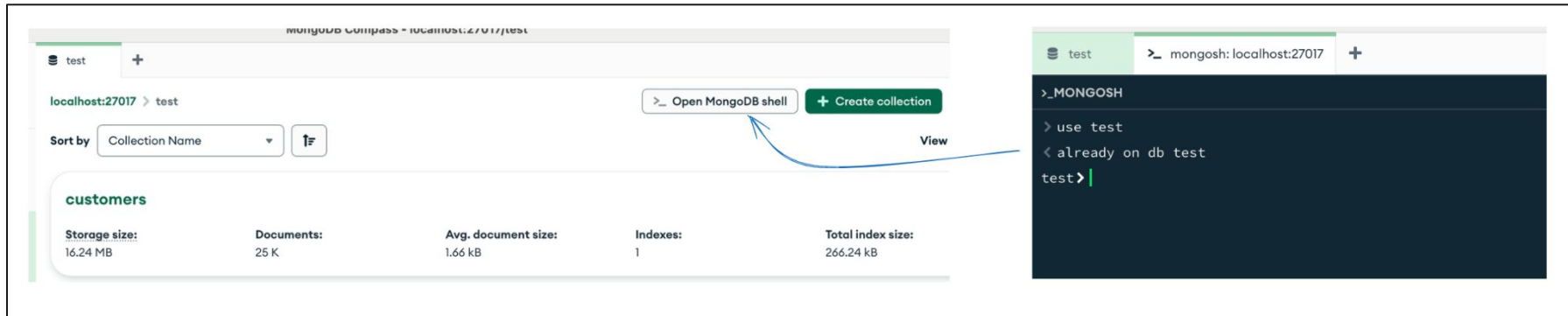
- Interfaces utilisateur (MongoDB Express & MongoDB Compass)



BD orientée Document : MongoDB

MongoDB : Installation et prise en main

- Commandes de base : MongoDB shell (***mongosh***) => sur MongoDB Compass



The image shows two side-by-side screenshots from a presentation. The left screenshot is of the MongoDB Compass web interface. It shows a database named 'test' connected to 'localhost:27017'. A collection named 'customers' is selected, showing 25 documents with a storage size of 16.24 MB. A button labeled '>_ Open MongoDB shell' is highlighted with a blue arrow pointing to the right. The right screenshot shows the MongoDB Shell (mongosh) interface. The prompt is '>_MONGOSH'. The user has entered 'use test', and the response is '< already on db test'. The prompt is now 'test>'. A blue arrow points from the 'Open MongoDB shell' button in the Compass interface to the shell window.

mongodb Compass - localhost:27017/test

test +

localhost:27017 > test

Sort by Collection Name

customers

Storage size:	Documents:	Avg. document size:	Indexes:	Total index size:
16.24 MB	25 K	1.66 kB	1	266.24 kB

>_ Open MongoDB shell + Create collection View

test >_ mongosh: localhost:27017 +

```
>_MONGOSH
> use test
< already on db test
test>
```



BD orientée Document : MongoDB

MongoDB : Installation et prise en main

- Commandes de base : MongoDB shell (*mongosh*) => sur MongoDB Compass

Database commands

```
// Show all databases
show dbs

// Switch to or create a new database
use myDatabase

// Show current database
db
```

Collection commands

```
// Show collections in the current database
show collections

// Create a new collection
db.createCollection("myCollection")

// Drop (delete) a collection
db.myCollection.drop()
```

BD orientée Document : MongoDB

MongoDB : Installation et prise en main

- Commandes de base : MongoDB shell (*mongosh*) : CRUD



```
// Insert one document
db.users.insertOne({ name: "Alice", age: 25 })

// Insert multiple documents
db.users.insertMany([
  { name: "Bob", age: 30 },
  { name: "Charlie", age: 35 }
])
```



```
// Find all documents
db.users.find()

// Find with filter
db.users.find({ age: { $gt: 30 } })

// Find one document
db.users.findOne({ name: "Alice" })
```

BD orientée Document : MongoDB

MongoDB : Installation et prise en main

- Commandes de base : MongoDB shell (*mongosh*) : CRUD

```
// Update one document
db.users.updateOne({ name: "Alice" }, { $set: { age: 26 } })

// Update multiple documents
db.users.updateMany({}, { $inc: { age: 1 } }) // Increment age for all users
```



```
// Delete one document
db.users.deleteOne({ name: "Bob" })

// Delete multiple documents
db.users.deleteMany({ age: { $lt: 30 } })
```





BD orientée Document : MongoDB

MongoDB : Installation et prise en main

- **Commandes de base : MongoDB shell (*mongosh*) : les filtres**
 - Utilisable à travers la méthode `.find()`

Operator	Description	Example
<code>\$eq</code>	Equal to	<code>{ age: { \$eq: 25 } }</code>
<code>\$ne</code>	Not equal to	<code>{ age: { \$ne: 25 } }</code>
<code>\$gt</code>	Greater than	<code>{ age: { \$gt: 18 } }</code>
<code>\$gte</code>	Greater than or equal to	<code>{ age: { \$gte: 18 } }</code>
<code>\$lt</code>	Less than	<code>{ age: { \$lt: 30 } }</code>
<code>\$lte</code>	Less than or equal to	<code>{ age: { \$lte: 30 } }</code>
<code>\$in</code>	In array of values	<code>{ age: { \$in: [18, 25, 30] } }</code>
<code>\$nin</code>	Not in array of values	<code>{ age: { \$nin: [18, 25, 30] } }</code>



BD orientée Document : MongoDB

MongoDB : Installation et prise en main

- **Commandes de base : MongoDB shell (*mongosh*) : les filtres**
 - Utilisable à travers la méthode `.find()`

Operator	Description	Example
<code>\$and</code>	Logical AND	<code>{ \$and: [{age: {\$gt:18}}, {age: {\$lt:30}}] }</code>
<code>\$or</code>	Logical OR	<code>{ \$or: [{age: {\$lt:18}}, {age: {\$gt:65}}] }</code>
<code>\$not</code>	Negate a condition	<code>{ age: { \$not: { \$gt: 18 } } }</code>
<code>\$nor</code>	None of the conditions are true	<code>{ \$nor: [{age: {\$gt:18}}, {name: "Alice"}] }</code>

BD orientée Document : MongoDB

MongoDB : Installation et prise en main

- Commandes de base : utilisation via Python

Install client and connect

```
!pip install pymongo dotenv
from pymongo import MongoClient
import os
from dotenv import load_dotenv, find_dotenv

load_dotenv(find_dotenv())

usr = "root" # os.environ.get("MONGO_INITDB_ROOT_USERNAME")
pwd = "pwd" # os.environ.get("MONGO_INITDB_ROOT_PASSWORD")
url = f"mongodb://{usr}:{pwd}@mongo:27017"
client = MongoClient(url)
```

Create database & collection

```
db = client["school"]
students = db["students"]
```

Insert document

```
new_student = {
    "name": "Alice",
    "age": 22,
    "class": "Math",
    "grades": [15, 17, 19]
}
insert_result = students.insert_one(new_student)
```

Read documents

```
# Get all students
all_students = students.find()
for student in all_students:
    print(student)

# Find one student
student = students.find_one({"name": "Alice"})
print("Found:", student)
```

Update

```
students.update_one(
    {"name": "Alice"},
    {"$set": {"age": 23}}
)

# Verify update
updated_student = students.find_one({"name": "Alice"})
print("Updated:", updated_student)
```

Delete

```
students.delete_one({"name": "Alice"})
print("Deleted Alice")
```



BD orientée Document : MongoDB

TP1 : Création et alimentation d'une BD MongoDB

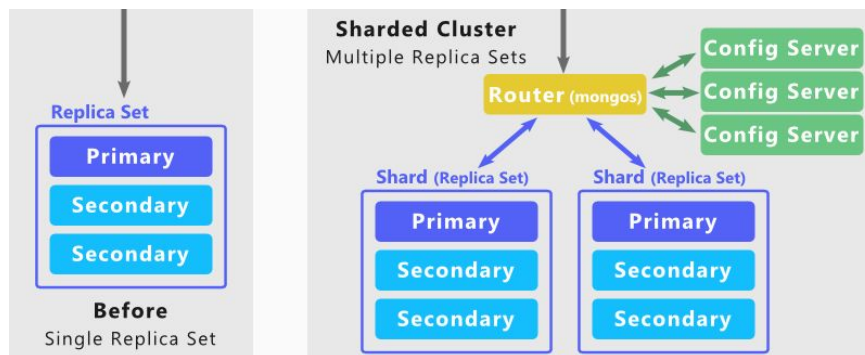
- **Base de données MongoDB :**

- Avec le terminal, se mettre dans le dossier
 - ***formation-bigdata/05-mongodb/***
- Lancer la commande :
 - ***python generate_etat_civil_json.py --actes 200000 --sortie ./data/actes.jsonl***
- Se mettre dans le sous dossier ci-dessous et copier le fichier *.env.example* pour le renommer à *.env* tout court:
 - ***01_prise_en_main/***
- Construire les conteneurs docker :
 - ***docker-compose up -d***
- Via un navigateur, aller à l'adresse : <http://localhost:8888/lab/>
- Suivre les instructions du notebook ***01_mongodb.ipynb*** dans le dossier notebooks

BD orientée Document : MongoDB

MongoDB : le passage à l'échelle

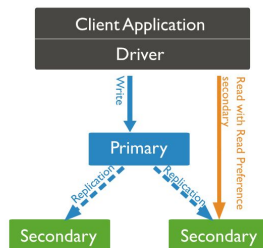
- Deux mécanismes distincts :
 - **Réplication** (replicaset) : la même donnée sur plusieurs serveurs • pour la **disponibilité**
 - **Partitionnement** (sharding) : des données différentes sur plusieurs serveurs • pour le **volume**



BD orientée Document : MongoDB

MongoDB : le passage à l'échelle

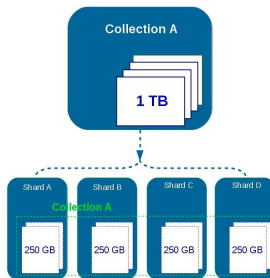
- Deux mécanismes distincts :
 - Réplication (replicaset)
 - Plusieurs (au moins 3) serveurs portant la même donnée : un primaire, plusieurs secondaires
 - Toutes les écritures passent par le primaire, qui les propage
 - Le primaire tombe : les survivants élisent un nouveau primaire,
 - C'est pourquoi il en faut un nombre impair : pour qu'une majorité se dégage



BD orientée Document : MongoDB

MongoDB : le passage à l'échelle

- Deux mécanismes distincts :
 - Partitionnement (sharding)
 - Les données sont réparties entre plusieurs serveurs selon une clé
 - Chaque serveur ne détient qu'une part, découpée en blocs
 - Un routeur dirige chaque requête vers les serveurs concernés
 - Chaque part peut être elle-même répliquée : partitionnement et réplication
 - NB : bien choisir la clé de partitionnement pour une bonne répartition





BD orientée Document : MongoDB

TP2 : Démo replicaset

- **Réplication des données :**
 - Suivre les instructions dans [05-mongodb/02_replicaset/docker-compose.replicaset.yml](#)



Part 3 :

Traitement distribué avec
Spark



3.1

Introduction à Spark



Introduction à Spark

- **Objectif :**
 - Comprendre l'architecture de Spark : qui fait quoi, où ?
 - Distinguer transformations et actions
 - Lire un plan d'exécution
 - Écrire ses premières transformations en PySpark



Introduction à Spark

Rappels sur Spark

- **Apache Spark :**
 - Système open source rapide qui traite de **grandes quantités de données** en utilisant la **mémoire (RAM)** plutôt que des disques durs lents.
- **Mode de fonctionnement :**
 - **Calcul distribué :** répartit le calcul sur plusieurs machines ou processus
 - **Exécution paresseuse :** n'exécute les calculs que quand c'est nécessaire
- **Différences avec Map Reduce :**
 - N'écrit pas nécessairement sur le disque, fait les calculs sur la mémoire RAM



Introduction à Spark

Rappels sur Spark

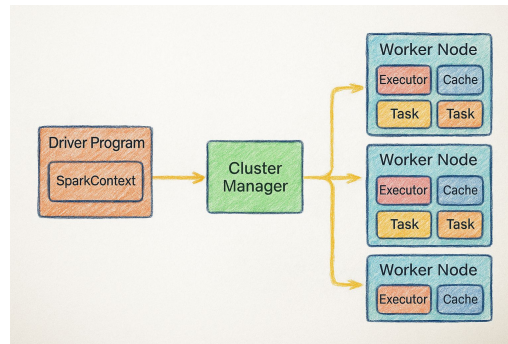
- **Apache Spark :**
 - Système open source rapide qui traite de **grandes quantités de données** en utilisant la **mémoire (RAM)** plutôt que des disques durs lents.
- **Mode de fonctionnement :**
 - **Calcul distribué :** répartit le calcul sur plusieurs machines ou processus
 - **Exécution paresseuse :** n'exécute les calculs que quand c'est nécessaire
- **Différences avec Map Reduce :**
 - N'écrit pas nécessairement sur le disque, fait les calculs sur la mémoire RAM



Introduction à Spark

Architecture avec Spark

- Une architecture en trois rôles:
 - Le **pilote (driver)** : votre programme. Il construit le plan et coordonne
 - Les **exécuteurs (executors)** : les processus qui font le travail, sur les machines du cluster
 - Le **gestionnaire de ressources (cluster manager)**: il attribue processeurs et mémoire aux exécuteurs
- Le pilote ne traite pas les données : il donne les ordres
- Les données sont dans les workers :
 - “déplacer le calcul vers la données”





Introduction à Spark

Architecture avec Spark

- **Même architecture, à toutes les échelles:**
 - Mode **local** : le pilote et les exécuteurs sont des **processus** de la même machine
 - Model **cluster** : pilote et exécuteurs sont répartis sur **plusieurs serveurs**
 - Le code ne change pas :
 - Seule la configuration de démarrage change
 - Cela permet de développer sur un portable et de déployer sur un cluster

Introduction à Spark

Architecture avec Spark

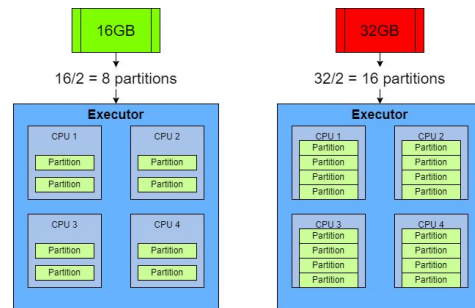
- L'unité de travail : les *partitions*

- Les données sont découpées en **partitions**
- Une **partition** est traitée par un **exécuteur**, indépendamment des autres
- Le parallélisme est plafonné par le nombre de partitions :

- Trop peu : les machines attendent
- Trop nombreux : le coût de coordination l'emporte
 - Analogie : comptage de bâtonnets par des élèves

- Astuce :

- viser des partitions de l'ordre de **100 à 200 Mo**,
- et au moins **autant de partitions** que de **cœurs disponibles**





Introduction à Spark

Fonctionnement de Spark

- Deux types de traitement : **Transformations** et **actions**
 - **Transformation :**
 - décrit un traitement : *select, filter, groupBy, join*
 - ne calcule rien : elle enrichit un plan
 - **Action :**
 - déclenche le calcul : *count, show, collect, write*
 - tant qu'aucune action n'est appelée, Spark n'exécute rien

Introduction à Spark

Fonctionnement de Spark

- Exécution paresseuse :
 - Avant un action, rien n'est calculé :
 - Spark voit tout le traitement avant de l'exécuter, et peut le réorganiser
 - Il ne lira que les **colonnes** nécessaires (*projection pushdown*)
 - Il appliquera les **filtres au plus tôt**, pendant la lecture (*predicate pushdown*)

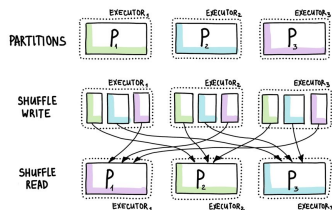
```
1 df.explain(True)
```

```
== Analyzed Logical Plan ==
saleDate: date, totalSales: double
Sort [totalSales#656 DESC NULLS LAST], true
+- Aggregate [saleDate#666], [saleDate#666, sum(cast((cast(quantity#665 as float) * price#669) as double)) AS totalSales#656]
   +- Filter (itemID#667 = 4)
      +- Join Inner, (itemID#667 = itemID#664)
         :- SubqueryAlias s
            :- SubqueryAlias spark_catalog.fmsandbox.sales
               :- Relation[itemID#664,quantity#665,saleDate#666] parquet
         +- SubqueryAlias i
            :- SubqueryAlias spark_catalog.fmsandbox.items
               :- Relation[itemID#667,itemName#668,price#669,effectiveDate#670] parquet
```

Introduction à Spark

Fonctionnement de Spark

- Le brassage (*shuffle*) : ce qui coûte cher
 - Certaines opérations exigent que des données **changent de machine**
 - C'est le cas d'un **groupBy**, d'un **join**, d'un **tri**
 - Toutes les valeurs d'une **même clé** doivent se retrouver au **même endroit**
 - Cela implique **écriture sur disque** et **transfert réseau** : c'est le coût dominant
 - Optimiser Spark, c'est d'abord réduire les *shuffles*





Introduction à Spark

Fonctionnement de Spark

- Les **modes** d'utilisation :
 - L'API **DataFrame** : `df.filter(...).groupBy(...).agg(...)`
 - Le **SQL** : `spark.sql("SELECT ... FROM ... GROUP BY ...")`
- Les **langages** interfacés :
 - Spark est écrit en Scala et tourne sur la machine virtuelle Java
 - Il s'utilise depuis **Python** (PySpark), **Scala**, **Java**, **R** et **SQL**
- Les **usages** :
 - **Spark SQL** : requêtes et traitement de tables
 - **Structured Streaming** : les mêmes opérations sur des flux continus
 - **MLlib** : apprentissage automatique réparti

Introduction à Spark

Fonctionnement de Spark

- Les usages :

Initialiser une session Spark

```
from pyspark.sql import SparkSession

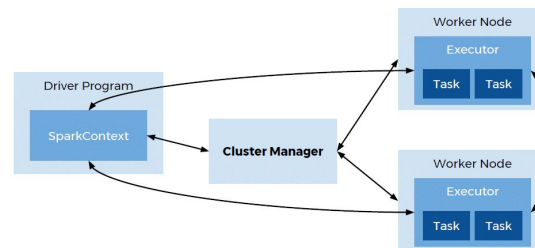
spark = SparkSession.builder \
    .appName("Optimized App") \
    .config("spark.executor.memory", "2g") \
    .config("spark.driver.memory", "1g") \
    .config("spark.executor.cores", "2") \
    .getOrCreate()
```

→ Mémoire allouée à chaque exécuteur

→ Mémoire pour le driver

→ Nombre de coeurs pour chaque executeur

Rappel de l'architecture





Introduction à Spark

Fonctionnement de Spark

- Les usages :

Charger un dataframe

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("Demo").getOrCreate()

df = spark.read.csv("sales.csv", header=True, inferSchema=True)

df.show()

df.groupBy("country").sum("amount").show()
```

Requêter avec Spark SQL

```
df.createOrReplaceTempView("sales")

spark.sql("""
SELECT country,
        AVG(amount)
FROM sales
GROUP BY country
""").show()
```



3.2

Principales fonctionnalités de Spark



Principales fonctionnalités de Spark

Manipuler des colonnes

- **Spark** travaille avec des dataframes, composés de colonnes et lignes
 - Les traitements se font principalement à travers des opérations sur les colonnes
- **Bibliothèque** de fonctions pour le traitement de la données :
 - *pyspark.sql.functions* contient plusieurs centaines de fonctions
 - Fonctions personnalisées possible : notion d'**UDF** (beaucoup plus lent)

<https://spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions.html>



Principales fonctionnalités de Spark

Manipuler des colonnes

- **Les fonctions courantes :**
 - **Sur du texte :** *trim*, *upper*, *regexp_replace*
 - **Conditions logiques :** *when / otherwise*, *between*
 - **Nettoyage :** *fillna*, *dropna*, *coalesce*
 - **Doublons :** *dropDuplicates* — avec ou sans liste de colonnes
 - **Types et dates :** *cast*, *to_date* avec le format attendu
 - **Agrégations :**
 - *groupBy(...)* accepte plusieurs colonnes
 - *agg(...)* produit plusieurs indicateurs en une seule passe
 - *when(...)* à l'intérieur d'un *sum* permet des comptages conditionnels



Principales fonctionnalités de Spark

Manipuler des colonnes

- Les fonctions courantes :
 - Les fonctions de fenêtrage :
 - Une agrégation classique réduit : dix lignes deviennent une
 - Une fenêtre conserve toutes les lignes, en ajoutant une valeur calculée sur un groupe
 - Définition des fenêtres :
 - *Window.partitionBy(...)* – sur quel groupe calculer
 - *Window.orderBy(...)* – dans quel ordre, si l'ordre compte
 - *rowsBetween(...)* – sur quelle étendue, pour les cumuls
 - Sans *partitionBy*, la fenêtre couvre toute la table : attention au volume

Principales fonctionnalités de Spark

Manipuler des colonnes

- Les fonctions courantes :
 - Les fonctions de fenêtrage : calcul des ventes cumulés par département

Employé	Département	Mois	Ventes
Alice	A	Jan	100
Bob	A	Fév	120
Claire	A	Mar	150
David	B	Jan	80
Emma	B	Fév	90



```
from pyspark.sql import window
from pyspark.sql.functions import row_number, sum

# Fenêtre
w = window
    .partitionBy("Département")
    .orderBy("Mois")
    .rowsBetween(window.unboundedPreceding, window.currentRow))

result = (df
    .withColumn("Ordre", row_number().over(
        window.partitionBy("Département").orderBy("Mois")))
    .withColumn("Cumul_Ventes", sum("Ventes").over(w))
)
```



Département	Mois	Ventes	Ordre	Cumul_Ventes
A	Jan	100	1	100
A	Fév	120	2	220
A	Mar	150	3	370
B	Jan	80	1	80
B	Fév	90	2	170

NB : Attention au shuffle induit par le fenêtrage



Principales fonctionnalités de Spark

Manipuler des colonnes

- Le cache :
 - Par défaut, Spark **recalcule** tout à chaque **action**
 - **.cache()** :
 - Conserve le résultat en mémoire pour les actions suivantes
 - Utile quand une même table sert à **plusieurs calculs**
 - Inutile — voire nuisible — si on ne s'en sert **qu'une fois**
 - **.persist()** :
 - Similaire à **.cache()**, mais permet également de choisir le niveau de stockage (mémoire et/ou disque)
 - Et à libérer avec **.unpersist()** quand on a fini



Principales fonctionnalités de Spark

TP1/2 : Prise en main et fonctionnalités de Spark

- **Manipuler des données avec Spark :**
 - Dans le dossier de travail, se mettre au niveau du sous-dossier **06-spark**
 - Lancer la commande (disponible dans le fichier **Dockerfile.spark** , ne pas oublier le "." à la fin):
 - `docker build -f Dockerfile.spark -t formation-spark:1.0 .`
 - Lancer la commande (disponible dans le fichier **01-docker-compose.yml**) :
 - `docker compose -f 01-docker-compose.yml up -d`
 - Ouvrir **JupyterLab** sur <http://localhost:8888> :
 - Aller dans le dossier du **notebooks** et suivre les instructions des notebooks :
 - "01_premiers_pas.ipynb"
 - "02_nettoyer_agreger.ipynb"



3.3

Optimisations & **ML sur Spark**



Principales fonctionnalités de Spark

Optimisations: les opérations coûteuses

- Les jointures :

Syntaxe

```
result = df1.join(df2, df1.id == df2.id, "inner")
```

ou

```
result = df1.join(df2, ["id"], "inner")
```

Type de jointure

Type	Description
Inner Join	Conserve uniquement les lignes correspondantes
Left Join	Toutes les lignes de gauche + correspondances
Right Join	Toutes les lignes de droite + correspondances
Full Outer Join	Toutes les lignes des deux tables
Cross Join	Produit cartésien
Left Semi Join	Lignes de gauche ayant une correspondance
Left Anti Join	Lignes de gauche sans correspondance



Principales fonctionnalités de Spark

Optimisations: les opérations coûteuses

- Les jointures :
 - Problématiques de **performances** :
 - Le **déséquilibre** : si une clé concentre l'essentiel des lignes, une seule machine travaille pendant que les autres attendent
 - La **petite table** : joindre un référentiel de quatorze lignes à des millions sans optimisation peut déclencher un **shuffle**
 - Optimisations possibles :
 - **.cache()** : si le résultat de la jointure est utilisé plusieurs fois
 - la **diffusion (broadcast)** : la petite table est envoyée à chaque machine

Principales fonctionnalités de Spark

Optimisations: les opérations coûteuses

- **Le partitionnement des données** : notion de **data skew**
 - Spark découpe les données en plusieurs partitions, stockées dans les exécuteurs
 - **Data Skew** : apparaît lorsqu'une ou plusieurs partitions contiennent beaucoup plus de données que les autres.



- **Conséquences** :
 - Un executor travaille beaucoup plus longtemps que les autres.
 - Les autres executors restent inactifs en attendant la fin du traitement.
 - Certaines actions déclenchent beaucoup de *shuffles* coûteux.
 - Risque d'erreurs mémoire (*OutOfMemoryError*).



Principales fonctionnalités de Spark

Optimisations: les opérations coûteuses

- **Le partitionnement des données :**
 - Bonnes pratiques
 - Choisir une **clé de partition** fréquemment utilisée dans les filtres.
 - Éviter
 - une seule **grosse partition**.
 - des milliers de **petites partitions**.
 - Utiliser le **salting** pour répartir les clés très fréquentes sur plusieurs partitions.
 - Ajuster le nombre de partitions avec **.repartition()** ou **.coalesce()** selon le cas.
 - **.repartition()** : Réduire ou augmenter le nombre de partitions
 - **.coalesce()** : réduire les partitions (utile parfois pour exporter en csv par ex.)



Principales fonctionnalités de Spark

Machine learning sur Spark

- **Spark MLlib** : Bibliothèque de machine learning d'Apache Spark
 - Permet de traiter des **volumes massifs** de données grâce au **calcul distribué en mémoire**
 - Principaux composants :
 - **Algorithmes** : Classification, régression, clustering et filtrage collaboratif.
 - **Préparation des données (Feature Engineering)** : Extraction, transformation, sélection et réduction de dimensionnalité.
 - **Pipelines** : Enchaînement cohérent d'étapes de préparation et d'apprentissage inspiré de scikit-learn.
 - **Utilitaires** : Algèbre linéaire et outils statistiques distribués (tests statistiques, ...)



Principales fonctionnalités de Spark

Machine learning sur Spark

- **Spark MLlib** : Bibliothèque de machine learning d'Apache Spark
 - **Fonctionnement** :
 - MLlib attend toutes les **variables explicatives** dans **une seule colonne**, sous forme de **vecteur**
 - Faisable avec ***VectorAssembler***
 - Les variables qualitatives passent d'abord par ***StringIndexer***, puis ***OneHotEncoder***
 - Pour le reste : du classique en ML, comme avec *scikit-learn*

Principales fonctionnalités de Spark

Machine learning sur Spark

- **Spark MLlib** : Bibliothèque de machine learning d'Apache Spark

```
from pyspark.sql import SparkSession
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.linalg import Vectors

# Start session
spark = SparkSession.builder.appName("BasicMLlib").getOrCreate()

# Sample DataFrame
data = spark.createDataFrame([
    (0.0, Vectors.dense([0.0, 1.1, 0.1])),
    (1.0, Vectors.dense([2.0, 1.0, -1.0])),
    (0.0, Vectors.dense([2.0, 1.3, 1.0])),
    (1.0, Vectors.dense([0.0, 1.2, -0.5]))
], ["label", "features"])

# Train logistic regression model
lr = LogisticRegression()
model = lr.fit(data)

# Prediction
predictions = model.transform(data)
predictions.select("features", "label", "prediction").show()
```



features	label	prediction
[0.0,1.1,0.1]	0.0	0.0
[2.0,1.0,-1.0]	1.0	1.0
[2.0,1.3,1.0]	0.0	0.0
[0.0,1.2,-0.5]	1.0	1.0



Principales fonctionnalités de Spark

Machine learning sur Spark

- **Spark MLlib** : Bibliothèque de machine learning d'Apache Spark
 - **Plus value de Spark** :
 - Pour **entraîner** :
 - Un échantillon bien tiré suffit (n'est ce pas, les statisticiens ?)
 - Pour **prédire** :
 - Il faut passer sur la totalité : et c'est là qu'un moteur distribué sert vraiment
 - **Attention** :
 - Beaucoup de méthodes classiques ne se distribuent pas bien
 - Les modèles réellement distribués -> ceux qui se décomposent : arbres, régressions linéaires (par algo de descente de gradient)



Principales fonctionnalités de Spark

TP3 : Optimisations et ML sur Spark

- Manipuler des données avec Spark (ne faire que le dernier point si les autres sont déjà faits) :
 - Dans le dossier de travail, se mettre au niveau du sous-dossier **06-spark**
 - Lancer la commande (disponible dans le fichier **Dockerfile.spark**, ne pas oublier le "." à la fin):
 - `docker build -f Dockerfile.spark -t formation-spark:1.0 .`
 - Lancer la commande (disponible dans le fichier **01-docker-compose.yml**) :
 - `docker compose -f 01-docker-compose.yml up -d`
 - Ouvrir **JupyterLab** sur <http://localhost:8888> :
 - Aller dans le dossier du **notebooks** et suivre les instructions du notebook :
 - "03_apparier_et_classifier.ipynb"



Part 4 :

Pipelines et orchestration



Pipeline et orchestration

- **Ce qu'on a vu jusqu'à présent :**
 - Comment stocker la donnée?
 - Comment la traiter?
- **Ce qui nous reste à voir : l'amont et l'aval**
 - Comment récupérer les données en temps réel (streaming) ?
 - Kafka
 - Comment automatiser un programme de traitement de données ?
 - Airflow
 - Comment restituer les analyses ?
 - Power BI/Superset



4.1

Traitement en temps réel avec
Kafka



Traitement en temps réel avec Kafka

- **Objectif :**
 - Comprendre à quel problème répond un système de messages
 - Le vocabulaire : sujet (topic), partition, décalage, groupe de consommateurs
 - Produire et consommer un flux
 - Brancher Spark sur ce flux

Traitement en temps réel avec Kafka

La problématique

- Plusieurs **sources** produisent des **données en continu** :
 - tablettes d'enquêteurs, caisses, opérateurs télécoms
- Plusieurs **services** veulent les consommer, à leur rythme
- Relier directement chaque **source** à chaque **destination** :
 - $N \times M$ liaisons à maintenir
- Et si le **consommateur** tombe, les données sont perdues





Traitement en temps réel avec Kafka

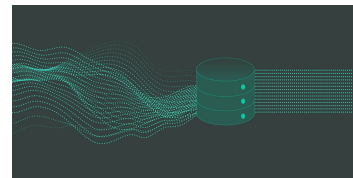
La problématique

Quelle solution actuellement utilisée par l'ANSD pour avoir des données en temps réel sur certaines problématiques (enquête, recensement, ...) ?

Traitement en temps réel avec Kafka

La problématique

- **Solution inadaptées :**
 - Un **fichier partagé** :
 - qui écrit, qui a déjà lu, comment lire pendant qu'on écrit ?
 - Une **base de données** classique :
 - conçue pour l'état courant, pas pour un flux continu à fort débit
- **Solution adaptée :**
 - Un **journal (log)**:
 - où l'on ajoute à la fin, et que chacun lit à son rythme
 - Analogie : un cahier où l'on écrit à la suite, jamais au milieu





Traitement en temps réel avec Kafka

Le modèle Kafka

- **Apache Kafka :**
 - Plateforme distribuée de streaming d'événements permettant de :
 - **collecter** des données **en continu** ;
 - **transporter** des messages **entre applications** ;
 - **stocker** temporairement des **flux d'événements** ;
 - **alimenter** des systèmes d'analyse en **temps réel**.
 - En résumé : système de messagerie capable de traiter des millions d'événements par seconde.



Traitement en temps réel avec Kafka

Le modèle Kafka

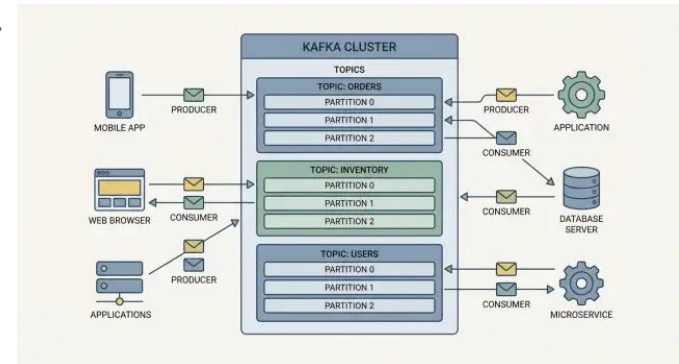
- **Fonctionnement de Kafka :**
 - Un **sujet (topic)** : un **journal** nommé, par exemple « actes-etat-civil »
 - On ajoute à la fin, on ne modifie jamais
 - Un **producteur** écrit des **événements** dans le topic, des **consommateurs** les lisent.
 - Les **événements** sont conservés — durée ou taille configurable
 - Ce n'est pas une file qui se vide : **on peut relire**
 - **Conséquence pratique** : un traitement bogué se rejoue sur les données passées

Producteurs		Kafka		Consommateurs
Application				
IoT	---->	Topic Commandes	---->	Spark
Logs	---->	Topic Logs	---->	Base SQL
Transactions	---->	Topic Paiements	---->	Dashboard

Traitement en temps réel avec Kafka

Le modèle Kafka

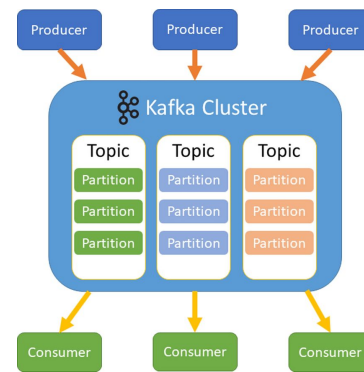
- Les concepts fondamentaux :
 - **Event (événement)** : donnée envoyée dans Kafka.
 - **Topic** : canal logique où sont stockés les événements.
 - **Producer** : application qui envoie des messages.
 - **Consumer** : application qui lit les messages.



Traitement en temps réel avec Kafka

Le modèle Kafka

- **Architecture interne Kafka :**
 - Un **cluster Kafka** contient plusieurs serveurs appelés **brokers**.
 - Chaque broker stocke une partie des données.
 - Un **topic** est découpé en **partitions** :
 - Kafka peut lire et écrire en parallèle.
 - L'ordre est garanti dans la partition et non entre partitions
 - Chaque **consommateur** mémorise son décalage :
 - Où il en est de sa lecture
 - **Legacy : ZooKeeper**
 - Service distribué de coordination qui organisait les brokers





Traitement en temps réel avec Kafka

Le modèle Kafka

- **Kafka et Spark :**
 - **Spark** lit un topic comme il lirait un fichier : même API, mêmes transformations
 - En flux : le traitement tourne en continu, par micro-lots (**micro-batch**)
 - En lot : on lit ce qui est disponible, puis on s'arrête
 - Une requête **en flux** ne se termine jamais toute seule
 - Dans un notebook, la cellule reste bloquée
 - Deux parades :
 - borner la durée
 - demander à Spark de s'arrêter quand il a tout lu



Traitement en temps réel avec Kafka

TP1 : Démo Kafka

- **Prise en main de Kafka :**
 - Suivre les instructions du README dans le dossier [07-kafka/00_demo_kafka](#)



Traitement en temps réel avec Kafka

TP2 : Début de pipeline avec Kafka/Spark/MongoBD

- Prise en main de Kafka :
 - Lancer la suite de conteneurs [07-kafka/01 kafka spark/docker-compose.yml](#)
 - Sur l'url du JupyterLab (<http://localhost:8888>), suivre les instruction du notebook :
 - [01 kafka spark/notebooks/01 kafka.ipynb](#)



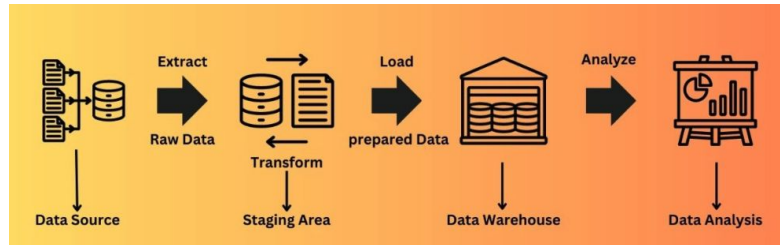
4.2

Orchestration de traitements avec **Airflow**

Orchestration de traitements avec Airflow

La problématique : automatisation

- Principales opérations dans un projet data : notion d'ETL (Extract - Transform - Load)



- Questions :
 - Comment lancer les **traitements automatiquement** ?
 - Comment respecter l'**ordre** des étapes ?
 - Comment **reprendre** après une **erreur** ?
 - Comment **suivre l'exécution** ?



Orchestration de traitements avec Airflow

La problématique : automatisation

Quelle solution utilisée par l'ANSD pour automatiser des traitements de données périodiques ?



Orchestration de traitements avec Airflow

La problématique : automatisation

- **Solutions classiques :**
 - Lancement manuel des scripts

```
python extraction.py  
python nettoyage.py  
python rapport.py
```

- **Inconvénients :**
 - intervention humaine ;
 - oublis ;
 - erreurs ;
 - aucune traçabilité.

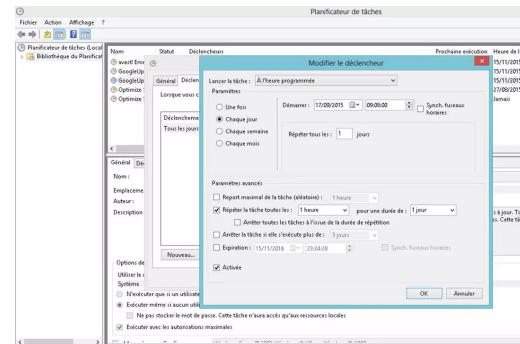
Orchestration de traitements avec Airflow

La problématique : automatisation

- Solutions classiques :
 - Tâches planifiées (cron, planificateur Windows, ...)

```
# syntax of cron
#
# |-----| sec (0 - 59)
# |-----| min (0 - 59)
# |-----| hour (0 - 23)
# |-----| day (1 - 31)
# |-----| month (1 - 12)
# |-----| weekday (0 - 6)
#
# * * * * * user-name command to execute
0 0 0 * * 6 root /scripts/have_fun
```

- Inconvénients :
 - Pas de gestion des dépendances entre traitements;
 - reprise difficile après une panne ;



Orchestration de traitements avec Airflow

La problématique : automatisation

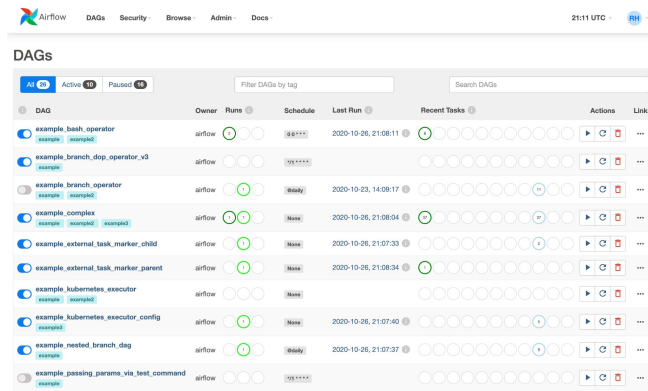
- Solutions modernes :
 - Les orchestrateurs
 - Lancement des tâches selon l'heure prévue et lorsque les dépendances sont satisfaites
 - En cas d'échec :
 - arrêt de la chaîne ;
 - possibilité de relancer uniquement la tâche en erreur ;
 - suivi de l'exécution.



Orchestration de traitements avec Airflow

La solution Apache Airflow

- Apache Airflow :
 - Plateforme d'orchestration de workflows;
 - Son rôle :
 - planifier les traitements ;
 - gérer les dépendances ;
 - lancer les tâches ;
 - suivre leur exécution ;
 - relancer automatiquement en cas d'échec.





Orchestration de traitements avec Airflow

La solution Apache Airflow

- **Fonctionnement d'Apache Airflow :**
 - Un **workflow** est représenté sous forme de **graphe orienté acyclique (DAG)**.
 - Écrit en Python : le DAG est du code, lu par Airflow -> le graphe est déduit et exécuté
 - Dans le workflow :
 - L'organisation est séquentielle par tâche : chacun attend le succès du précédent pour se lancer
 - Le décorateur **@dag** définit le graphe, **@task** définit une tâche
 - Plusieurs paramètre pour le dag :
 - **schedule** défini la fréquence de lancement (accepte une expression **cron**)
 - **start_date** : à partir de quand le calendrier existe
 - **catchup** : booléen disant s'il faut rattraper les exécutions passées ?

Orchestration de traitements avec Airflow

La solution Apache Airflow

- **Fonctionnement d'Apache Airflow :**
 - Un **workflow** est représenté sous forme de **graphe orienté acyclique (DAG)**.

```
from airflow.sdk import dag, task
from datetime import datetime

@dag(
    schedule="@daily",
    start_date=datetime(2026, 1, 1),
    catchup=False,
)
def pipeline_donnees():

    @task
    def extraction():
        print("Extraction")

    @task
    def nettoyage():
        print("Nettoyage")

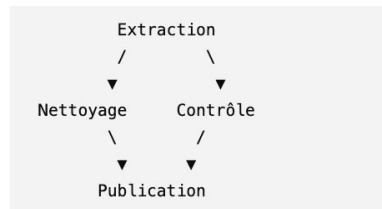
    @task
    def controle():
        print("Contrôle qualité")

    @task
    def publication():
        print("Publication des résultats")

    extraction() >> [nettoyage(), controle()] >> publication()

pipeline_donnees()
```

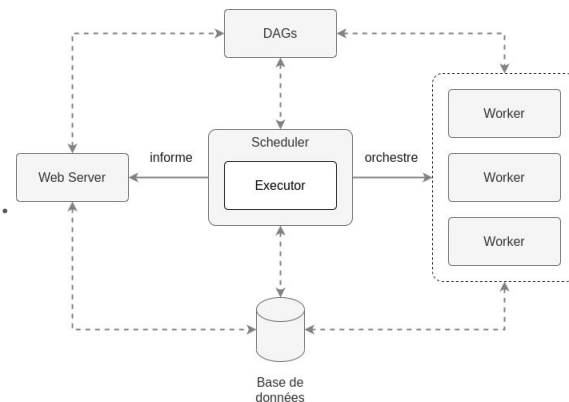
Représentation du DAG



Orchestration de traitements avec Airflow

La solution Apache Airflow

- Les composants d'Apache Airflow :
 - **Scheduler** : décide quelles tâches doivent démarrer.
 - **Executor** : exécute les tâches.
 - **Web server UI** : permet de suivre les workflows.
 - **Base de données** : conserve l'historique des exécutions.

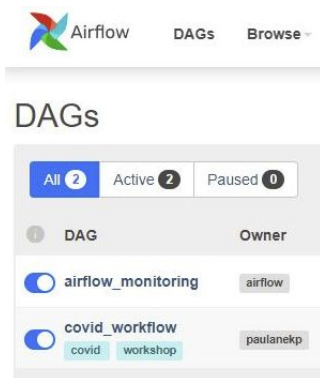


Orchestration de traitements avec Airflow

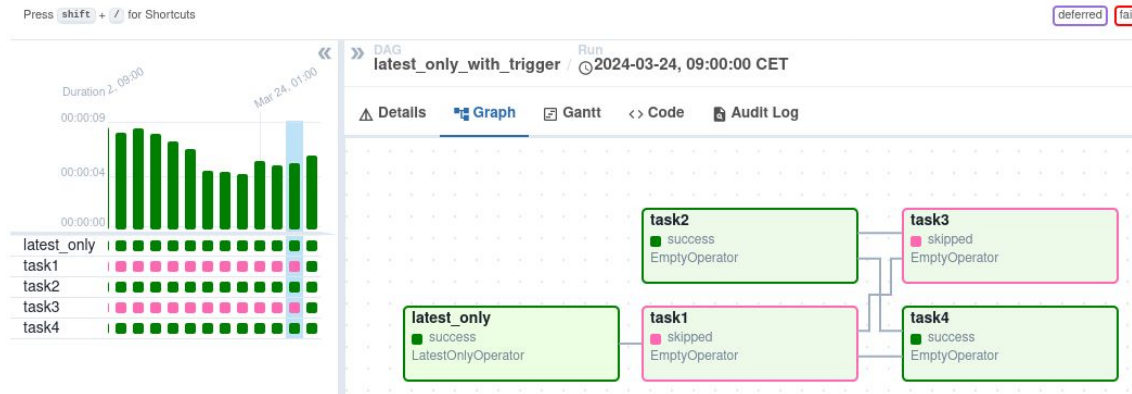
La solution Apache Airflow

- Les composants d'Apache Airflow :
 - Web server UI : permet de suivre les workflows.

Liste des DAGs



Détails d'un DAG





Orchestration de traitements avec Airflow

La solution Apache Airflow

- **Notions complémentaires :**
 - **Reprise et idempotence :**
 - Une tâche qui échoue est **relancée automatiquement**, selon la politique définie
 - Donc une tâche peut s'exécuter plusieurs fois sur la même période
 - Elle doit produire le même résultat à chaque fois (si une tâche est relancée après un échec, on ne doit pas obtenir un résultat différent ou des données en double.)
 - **Ce qu'Airflow ne fait pas :**
 - Ce n'est pas un moteur de traitement : il déclenche, il ne calcule pas
 - Ce n'est pas fait pour du temps réel : l'unité est la minute, pas la seconde
 - Ce n'est pas un outil de transfert de données entre tâches



Orchestration de traitements avec Airflow

TP : Orchestrer avec Airflow

- **Prise en main de Airflow :**
 - Suivre les instructions du guide : [08-airflow/GUIDE_TP.md](#)



Part 5 :

Analyse & déploiement



Analyse & déploiement

- Ce qu'on a vu jusque là :
 - Les données arrivent **en continu** et sont **conservées** : **MongoDB**
 - Un **traitement périodique** calcule les indicateurs : **Spark**, déclenché par **Airflow**
 - Ils sont publiés dans une table : **PostgreSQL**
 - Il reste à **les montrer** :
 - **Next step** : mise en place de **tableau de bord**



Analyse & déploiement

Pourquoi une couche de restitution

- Un **outil de visualisation** interroge une **table**, pas un flux ni des documents
- Il attend des **colonnes typées**, des **volumes modestes**, des **réponses rapides**
- D'où PostgreSQL :
 - quelques milliers de lignes agrégées, au lieu de millions de documents
 - MongoDB conserve ce qui est arrivé, PostgreSQL publie ce qu'on en a tiré
- C'est le **5ème V** du Big data : la valeur
 - Le Big Data permet de stocker et traiter les données ; la visualisation permet de leur donner du sens.



Analyse & déploiement

Les outils de restitution

- Plusieurs outils, différents surtout par le pricing et quelques options :
 - Tableau
 - Power BI
 - Looker
 - Superset
 - ...
- Celui que nous allons explorer :





Analyse & déploiement

TP : Faire un tableau de bord avec Apache Superset

- **Prise en main de Apache Superset :**
 - Suivre les instructions du guide :
 - [09-visualisation/GUIDE TP.md](#)



Conclusion



Synthèse de la formation

- Partie 1 — les limites d'un poste, les moteurs, les formats
- Partie 2 — les architectures, les conteneurs, le modèle document
- Partie 3 — le traitement distribué, l'appariement, l'apprentissage
- Partie 4 — les flux, l'orchestration
- Partie 5 — la restitution, et vos projets



Synthèse de la formation

- Mesurer avant de choisir — vous savez le faire, sur vos propres données
- Le format pèse plus que le moteur — et il vous survivra
- La façon d'écrire le calcul pèse plus que l'outil retenu
- Le choix des variables pèse plus que le choix de l'algorithme
- Ne distribuez que lorsque la mesure l'exige



Merci

Vos retours comptent : ce qui a manqué, ce qui a été trop rapide. Ne pas hésiter à les partager